Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Monterrey

School of Engineering and Sciences

Comparison between Single-Objective and Multi-Objective Algorithms for the creation of stable student groups.

A thesis presented by

Miguel Ángel Bravo Vidales

Submitted to the
School of Engineering and Sciences
in partial fulfillment of the requirements for the degree of

Master of Science

in

Computer Science

Monterrey, Nuevo León, December, 2019

Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Monterrey

School of Engineering and Sciences

The committee members, hereby, certify that have read the thesis presented by Miguel Ángel Bravo Vidales and that it is fully adequate in scope and quality as a partial requirement for the degree of Master of Science in Computer Science.

Dr. Alejandro Rosales Perez
Tecnológico de Monterrey
Principal Advisor

Dr. Santiago Enrique Conant Pablos
Tecnológico de Monterrey
Committee Member

Ramon Felipe Brena Pinero

Committee Member

Saúl Zapotecas Martínez
Universidad Autónoma Metropolitana (UAM)
Committee Member

Dr. Hugo Terashima Marin
Program Director

Monterrey, Nuevo León, December, 2019

Declaration of Authorship

I, Miguel Ángel Bravo Vidales, declare that this thesis titled, ”Comparison between Single-Objective and Multi-Objective Algorithms for the creation of stable student groups.” and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this dissertation is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Miguel Ángel Bravo Vidales
Monterrey, Nuevo León, December, 2019

©2019 by Miguel Ángel Bravo Vidales
All Rights Reserved

Dedication

(Only 1 page) Thanks for all your unconditional confidence, support, patience, and encouragement. You were my main motivation for pushing through this work.

Acknowledgements

(1 suggested page, no more than 2 pages) I would like to express my deepest gratitude to all those who have been side by side with me, along the long, but also short hours (paradox?), at night.

Do not forget the Tecnológico de Monterrey support on tuition and CONACyT with the support for living.

Comparison between Single-Objective and Multi-Objective Algorithms for the creation of stable student groups.

by
Miguel Ángel Bravo Vidales

Abstract

The following work finds solutions for the problem of making stable student groups, which consists in the grouping of different students for the practice of foreign languages through their level of experience and their topics of conversation of interest. It is the existence of groups where students practice the language in question, but having different possible ways of grouping students at a given time, there are also some better ways than others, according to predefined criteria. For these two possible scenarios are approached. In the first one, the problem is approached as one of single-objective optimisation or of general optimisation where it is intended to maximise the aforementioned criteria taking as weights of the same function. The second scenario is to address the problem as one of multi-objective optimisation, where tests with different genetic algorithms of the existing ones will be carried out to solve this problem. Finally, different comparisons will be made between the implementation of the mono-objective problem concerning the results obtained from the tests using multi-objective algorithms through a tool known as jMetal and another named JAMES which implements within it some of the most common genetic algorithms, in addition to some of the tools necessary to measure the performance of both algorithms to solve single-objective problems and to solve multi-objective problems. Bearing in mind that the result probably tends to be better with a multi-objective solution, it will also be determined which of these algorithms turns out to have a better performance for this specific problem.

The document conforms to the definition of the problem where both the origin of the problem and its technical specification are discussed to solve it. Then several objectives are listed that seek to solve the specified problem. In the next section, we talk about the hypothesis and the questions that you intend to answer if it is correct. Then, in the theoretical framework, some of the tools, methodologies, and related work that can be used to carry out the research are specified. In the methodology section, the steps that are carried out to carry out the experimentation are described in more detail. How the results will be evaluated and finally in the work plan the time needed to cover the aforementioned objectives is briefly described.

List of Figures

1.1 Solution model proposed to compare the approaches to solve SSGP. 5

2.1 Demonstration of the (HV) in two dimensions, the grey area represents the value of $HV(S, r)$ 20
2.2 Graph representing the Spread metric. 22
2.3 Graph representing the subspace of which Epsilon is based. 23
2.4 Example of distance between Reference Z, Solution A and Solution B. . . . 24
2.5 Example of distance between Reference Z, Solution A and Solution B, here the solutions from set B appear to be closer to Z. 25
2.6 A graph representing the main idea behind ($IGD+$). 25
2.7 An example of the Facebook interests ontology. 28

3.1 This is an example of a data set of nine users, each with their experience, three different interests and a participation percentage. 31
3.2 An example of the Facebook interests ontology. 33
3.3 An example of a mutation of a solution A. 36
3.4 Example of a crossover operation. 37

4.1 Performance of the LS algorithm. 48
4.2 Performance of the single objective GA. 48
4.3 Performance of the NSGA-II. 49

5.1 Reference front for the $n = 20$ problem. 51
5.2 Reference front for the $n = 200$ problem. 52
5.3 Reference front for the $n = 2,000$ problem. 52

List of Tables

4.1 Sample interests by student. 41
4.2 Comparison between the sample students. 42
4.3 Quantity of silences and their proportion by meeting. 43
4.4 Protagonism proportion for the Meeting IS1003d. 43
4.5 Protagonism proportion for the Meeting ES2008d. 43
4.6 Results of the conversation analysis. 44
4.7 The distribution by each level. 45
4.8 The first 20 Interests and their proportion. 46
4.9 Results for the Participation Analysis. 46

5.1 The results of the parameters experiment of $n = 20$ for SOA, by combinations of a population size p and a number of steps or generations s. 53
5.2 The results of the parameters experiment of $n = 200$ for SOA, by combinations of a population size p and a number of steps or generations s. 53
5.3 The results of the parameters experiment of $n = 20$ for MOA, by combinations of a population size p and a number of steps or generations s. 54
5.4 The results of the parameters experiment of $n = 200$ for MOA, by combinations of a population size p and a number of steps or generations s. 54
5.5 The v_{min} and v_{max} values for each objective function used in the normalisation. 55
5.6 Specific details of the Single-Objective Algorithms Implementations based on the jMetal Framework. 56
5.7 Specific details of the Single-Objective Algorithms Implementations based on the James Framework. 57
5.8 Specific details of the Multi-Objective Algorithms Implementations. 58
5.9 The boundaries for each problem size and each function based on the best found results 60
5.10 60
5.11 Median and Interquartile Range for the IGD+. Metric using Thumb-Parameters 61
5.12 Median and Interquartile Range for the EP. Metric using Thumb-Parameters . 62
5.13 Median and Interquartile Range for the SPREAD Metric using Thumb-Parameters 62
5.14 Median and Interquartile Range for the HV. Metric using Thumb-Parameters . 62
5.15 Friedman Ranking on IGD+ metric using Thumb-Parameters 63
5.16 Friedman Ranking on EP. metric using Thumb-Parameters 64
5.17 Friedman Ranking on SPREAD metric using Thumb Parameters 65
5.18 Friedman Ranking on HV metric using Thumb Parameters 65
5.19 Number of stable students in the groups by preference criteria 66

5.20 Results of the Pearson correlation analysis 66

Contents

Abstract **v**

List of Figures **vi**

List of Tables **viii**

1 Introduction **1**
1.1 Research Context 1
1.2 Problem Definition 2
1.3 Objectives 2
1.4 Hypothesis 3
1.5 Contributions 4
1.6 Solution Model 4
1.7 Document Structure 5

2 Background **7**
2.1 Optimality 7
2.1.1 Single-Objective Optimisation 7
2.1.2 Multi-Objective Optimisation 8
2.2 Evolutionary Algorithms 8
2.3 Single-Objective Optimisation Algorithms 10
2.3.1 Local Search 10
2.3.2 Genetic Algorithms 10
2.3.3 Evolution Strategies 11
2.3.4 Random search 12
2.3.5 Random Descent 12
2.3.6 Parallel Tempering (Replica Exchange Monte Carlo) 13
2.4 Multi-Objective Algorithms 14
2.4.1 Multi-objective Random Search 14
2.4.2 Electrostatic Potential Energy Evolutionary Algorithm 14
2.4.3 Many Objective Metaheuristic Based on the $R2$ indicator 15
2.4.4 Non-dominated Sorting Genetic Algorithm II 16
2.4.5 Strength Pareto Evolutionary Algorithm 2 17
2.5 Comparing Single-Objective and Multi-Objective Optimisation 19
2.6 Metrics 20

2.6.1 Hypervolume . . . 20
2.6.2 Spread . . . 21
2.6.3 Epsilon . . . 22
2.6.4 Generational Distance . . . 23
2.6.5 Inverted Generational Distance . . . 24
2.6.6 Inverted Generational Distance + . . . 24
2.7 Frameworks . . . 26
2.7.1 jMetal . . . 26
2.7.2 James . . . 26
2.8 Preference Criteria . . . 27
2.8.1 Benne and Sheats functional roles . . . 27
2.8.2 Facebook Ad Platform . . . 27
2.8.3 CEFR . . . 28

3 Methodology **30**
3.1 Problem Design . . . 30
3.2 Objective Functions . . . 31
3.2.1 Group Size . . . 32
3.2.2 Experience level . . . 32
3.2.3 Interests Similarity . . . 32
3.2.4 Participation style . . . 33
3.3 Stability . . . 34
3.4 Operators . . . 35
3.4.1 Initialisation . . . 35
3.4.2 Mutation . . . 36
3.4.3 Crossover . . . 37
3.4.4 Repair . . . 38
3.5 Discussion for the following chapter . . . 39

4 Preliminary Experiments **40**
4.1 Experiments regarding objective functions . . . 40
4.1.1 Interests distance function . . . 40
4.1.2 Participation style Function . . . 41
4.2 Synthetic data-set creation . . . 44
4.2.1 Experience Level . . . 44
4.2.2 Interests . . . 45
4.2.3 Participation style . . . 46
4.3 Exploratory Experiments . . . 46
4.3.1 Single-Objective Optimisation . . . 47
4.3.2 Multi-Objective Algorithm . . . 49
4.4 Preliminary conclusions . . . 49

5 Experiments and Results **50**
5.1 Experiments Setup 50
5.1.1 Searching for a Reference Pareto Front 51
5.1.2 Searching for Reference Parameters 52
5.2 Implementation Details 54
5.2.1 Single-Objective Algorithms 54
5.3 Scalarization Methods 58
5.3.1 Weighted Sum Method 59
5.3.2 Chebyshev Scalarization 59
5.3.3 ϵ-constraints Scalarization 59
5.4 Experimental Phases 59
5.4.1 Scalarization Weights 60
5.4.2 First phase: Thumb Parameters 61
5.4.3 Second Phase 65
5.4.4 Stability evaluation 65
5.5 Correlation between the results 66

6 Conclusions and future Work. **67**
6.1 Conclusion 67
6.2 Future work 68

Bibliography **74**

Chapter 1

Introduction

1.1 Research Context

As the world becomes increasingly globalised, the content produced in languages other than that of the mother tongue increases, especially that in the English language, which turns out to be the dominant worldwide. The need to learn this language dramatically increases as time progresses, and traditional teaching methods do not provide enough for the entire population that needs to learn it. It is thus necessary to create alternatives that are more accessible and optimal for students who are often occupied by a very active pace of life and have little flexibility in their schedule to practice among their acquaintances.

One alternative to help learn a new language consists of a system in which students who do not know each other are grouped according to their topics of interest. In this manner, students practice the use of language according to having conversations about these interests and a similar level of experience, allowing all participants to understand and take part.

This thesis aims to determine an optimal manner to group students into different virtual groups so that they can improve their use of language and not be frustrated either because of the unattainable difficulty or because the topics do not suit their liking. There are several aspects that can make one grouping better than another; for example, the level of language proficiency must be relatively compatible, that is, very advanced students cannot be put together with beginners, as the advanced student gets bored with the difficulties of the beginner, and the latter does not understand what the advanced say. Additionally, it is sought that the interests of the students are compatible, in order to generate conversations of interest to all. For example, while some students may have sports among their subjects of interest, some prefer football and others prefer swimming. Another criterion is the intervention style of each student, which can be more or less active and more or less shy. Thus, if many shy and passive students are in the same room, it will be difficult to have an agile dynamic that makes the session entertaining. For this reason, it is proposed to address the problem as one of multi-objective optimisation, where the topics of conversation and the level of the student experience are the objectives to optimise. To this end, a mono-objective solution that combines the above criteria in a weighted aggregate function will be implemented. Then a comparison will be made of the results obtained with different genetic algorithms based on the mono-objective solution and

the multi-objective solution. It is believed that through the use of a multi-objective strategy, the student's experience will improve significantly compared to existing methods of teaching and practising foreign languages.

1.2 Problem Definition

The problem is defined as a variation of the Stable Roommate Problem (SRP), which is a variation of the Stable Marriage Problem (SMP) [31]. Only in this case, individuals do not know each other, so they have no real preference; nevertheless, the preference is predicted based on the similarity of the level of experience in the foreign language of the student and the interest that is shared with the rest of the group. The above is based on the fact that these two factors directly affect the student's preference to belong to the group.

In addition to this, group size and participation styles are also considered as restrictions for the benefit of the rest of the system, as a person may prefer to belong to a group that has more in common with him/her; however, his/her way of participating can alter the behaviour of the rest of the group. For example, an outgoing student who participates too much in a conversation can prevent the rest of the group from participating; conversely, a group in which no one participates needs an outgoing student to initiate the conversations.

In the same way, the size of the group is defined dynamically to obtain homogeneous participation among its members. It is defined dynamically since the absence or presence of some students can affect the performance of the group, although its size is not considered ideal.

This research aims to obtain the most stable solution possible concerning the predicted preferences of students within a reasonable time. Although there are many examples and variations of SRP in the literature, there is no characterisation that adapts directly to the problem defined considering the groups as variables. SRP is considered as a NP-complete problem, so it is relevant to find a solution in different by the use of heuristics and meta-heuristics. A comparison between single-objective algorithms and multi-objective is suggested to find a good enough solution for this problem.

1.3 Objectives

Compare single-objective and multi-objective strategies to find stable studying groups based on their interests and levels of experience, and consider the group size and their participation as constraints.

The specific objectives that this entails are described below:

1. Define stability and near-stability for the problem.

2. Build an artificial data-set to compare the potential approaches.
3. Define the relevant attributes and parameters to formulate the single and multi-objective functions.
4. Implement mono-objective strategies to generate solutions to the problem, which will be the basis for comparing the multi-objective solutions that are made.
5. Implement and test different multi-objective strategies and compare them with the results obtained with the mono-objective strategies.
6. Review the results and make a report indicating if a better result was reached with a multi-objective strategy and what was the algorithm that had the best results.

1.4 Hypothesis

Multi-Objective algorithms can outperform single-objective algorithms in finding near-stable studying groups based on their interests and level of experience, while considering the group size and participation as constraints.

This research tries to respond the following questions:

- How stable are the solutions found?
- Which advantages multi-objective algorithms may have against single-objective algorithms and vice-versa?
- Is optimisation even necessary? (Can a random solution be improved using optimisation techniques?)
- How does single-objective solutions are, compared with multi-objective ones?
- Can a single-objective algorithm have a solution that belongs to the Pareto front?
- Which single-objective algorithm outperforms the other single-objective algorithms for this problem?
- Which multi-objective algorithm outperforms the other multi-objective algorithms for this problem?
- Which algorithm results to be the best overall?

1.5 Contributions

Below is shown a list of the main contributions of this research. They were established to achieve the objectives and answer the questions previously raised.

- Introduces the problem of Stable Student Groups (SSGP), based mostly in SMP and SRP and other similar combinatorial problems.
- Makes a comparison of optimisation of different preference criteria against the stability of the resulting groups.
- Establishes a way to solve SSGP with unknown preferences, using a predicted preferences based on a preference criteria.
- Defines a set of objective functions to help predict the preference of the students of a given data-set.
- Builds a synthetic data-set based on distribution data, determined by the objective functions previously defined.
- Defines a set of operators to be used for SSGP, based on small but significant changes to the groups.
- Presents different models for the solution of SSGP, and a way to compare them.
- Presents an extensive analysis of the solutions found making use of different ways to compare single-objective algorithms to multi-objective algorithms in a ways that are considered fair.

1.6 Solution Model

The solution for the established SSGP follows a structure for optimisation using evolutionary algorithms. First, the objective functions that represent the objective of the problem are defined. Then a synthetic database is generated using the distribution data for several sources. This data will be evaluated by the objective functions. After that, the operators for the evolutionary algorithms are defined and tested.

Finally, single-objective algorithms and multi-objective algorithms are compared within them and with each other. These comparisons follow closely the one based on the one developed by Ishibuchi et al. [30], who compared single-objective algorithms with multi-objective algorithms. A diagram of the solution model can be seen in Figure 1.1.

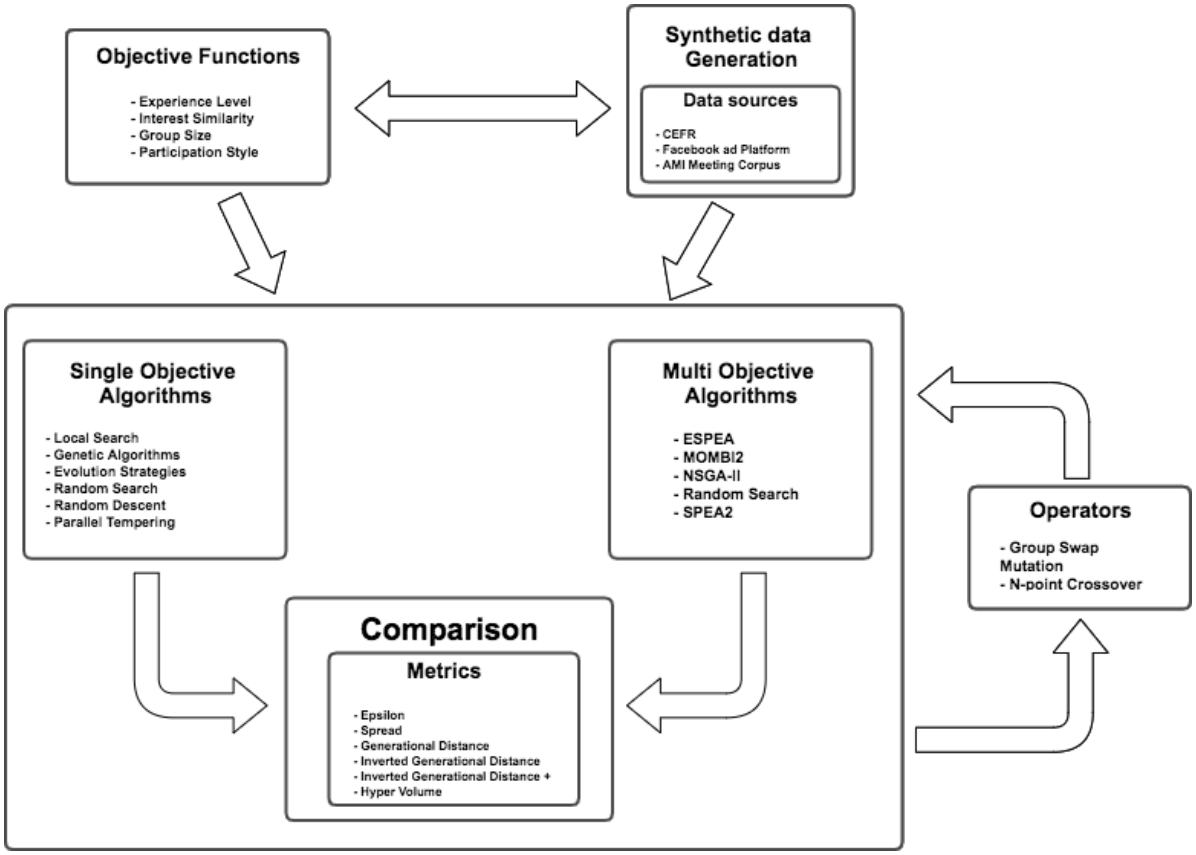


Figure 1.1: Solution model proposed to compare the approaches to solve SSGP.

1.7 Document Structure

Including this introduction, the thesis consists of 6 chapters.

Chapter 2 provides a review of the literature. Includes basic definitions for the single-objective algorithms and multi-objective algorithms to be used. Additionally, it includes some background for performing a comparison between them, including the different metrics used. This chapter also discusses the preference criteria, which is later used to define the objective functions. Finally, it includes a brief introduction of the frameworks used to evaluate the different algorithms.

Chapter 3 presents the design of the problem and the approach to solving it. It defines the objective functions to use and the operators for the different algorithms.

Chapter 4 shows a set of preliminary experiments. Beginning to test the defined objective functions. Then, a brief methodology for the data-set creation is explained and later tested with a set of exploratory experiments.

Chapter 5 contains the main experiments and a discussion of their results. The experiments are separated in the setup and three phases. The setup introduces the parameters to be used for the rest of the experimentation. The first phase contains the first set of experiments using the parameters defined in the setup. Second and Third phases are the result of the feedback obtained during the first phase.

Chapter 6 explains the conclusions of the research and mentions the future work that can be derived.

Chapter 2

Background

This chapter presents a review of the literature for the considered single-objective algorithms and multi-objective algorithms. It also defines important concepts to understand the results produced, such as the concept of optimisation and dominance. After, it introduces the metrics that help compare the different approaches. Finally, it explains the criteria considered for preference prediction.

2.1 Optimality

There are two interpretations of optimality. One refers to single-objective optimisation and the other to multi-objective optimisation.

2.1.1 Single-Objective Optimisation

An optimisation problem refers to find the solution that gets the minimum or maximum value of a given function $f(x)$, subject to a set of constraints. Mathematically, it can be stated as follows:

$$\begin{aligned} \min f\left(\boldsymbol{x_i}\right) \\ \text{subject to } g_i\left(\boldsymbol{x_i}\right) \leq 0 \\ h\left(\boldsymbol{x_i}\right) = 0 \\ x_i \in \Omega \end{aligned} \tag{2.1}$$

where $\vec{x} = (x_1, ..., x_n)$ is an n-dimensional decision variable vector, $g_i(x_i)$ and $h_j(x_i)$ refer to the set of constraints that must be fulfilled while optimising $f(x_i)$, and Ω is the universe representing all the acceptable solutions possible that can be used to satisfy $f(x_i)$ and its constraints. The $\vec{x}$ decision vector can be whether continuous or discrete variables and its evaluation can also be continuous or discrete.

Definition 1. *Single-objective global optimisation is defined as given a function $f : \omega \subseteq \mathbb{R}^n \to \mathbb{R}, \Omega \neq \emptyset$ for $x_i \in \Omega$ the value $f^* \triangleq f(\vec{x}^*) > -\infty$ is called a* ***global minimum*** *if and*

only if

$$\forall x \in \Omega : f(\vec{x}^*) \leq f(x). \tag{2.2}$$

$\vec{x}^*$ is by definition the global minimum solution. The goal of determining the global minimum solution is called the **global optimisation problem** for a single-objective problem.

2.1.2 Multi-Objective Optimisation

Optimisation problems that involve two or more objectives are known as multi-criteria or multi-objective optimisation problems (MOPs). In MOPs, objectives are usually in conflict, this means that it is not possible to compare two solutions $\vec{x}$ and $\vec{y}$ directly, as there is no single solution that would simultaneously be the best for all objectives. The quality of the solutions is therefore compared by their dominance.

Definition 2. *The **Dominance** of a solution a_i of a set $A = (a_1, ..., a_n)$ is said to dominate a solution b_i of another set $B = (b_1, ..., b_n)$, denoted by $a_i \preceq b_i$ or $f(a) \preceq f(b)$ with, if and only if a_i is at least as optimal as b_i in all objectives of b_i and with at least one of its objectives better than b_i [4].*

In multi-objective optimisation, the goal is to find the set of optimal solutions, known as the **Pareto Optimal Set**. Pareto optimality is defined concerning the concept of non-domination between the points of the objective space Ω if and only if there is no vector in the set A that results better in set B.

Definition 3. *A **Pareto Optimal Set** (POS) is composed of optimal solutions, which its internal objectives cannot be improved simultaneously. Formally defined as:*

$$\mathrm{POS} := \{\mathrm{x}^* \in \mathcal{X} : \nexists \mathrm{x} \in \mathcal{X} \subset \mathbf{R}^n, \mathrm{x} \prec \mathrm{x}^*\} \tag{2.3}$$

Definition 4. *The **Pareto Optimal Front** (PF) is the corresponding objective space of vectors from the **Pareto optimal set** termed as non-dominated. This collection of Pareto optimal solutions being the result of the same MOP, where:*

$$PF^* := \{u = F(x) | x \in P^*\} \tag{2.4}$$

2.2 Evolutionary Algorithms

Evolutionary algorithms (EAs) are stochastic optimisation techniques inspired by Darwin's theory of evolution. An EA uses mechanisms inspired by biological evolution such as reproduction, mutation, recombination, and selection. The candidate solutions for optimisation are part of a population, and a fitness function determines the quality of each of these solutions. This fitness takes part after different iterations or generations in which it is sought to get as

close as possible to the global optimum [20].

Based on this fitness, the best candidates are more likely to be chosen as the parents for the next generation by applying recombination and mutation operators. Recombination is an operator that produces new candidates (children) solutions by combining the genetic information of two or more selected candidates (parents). A mutation operation applies small changes to a solution. Executing recombination and mutation leads to a set of new candidates (the offspring) that compete based on their fitness for a place in the next generation. This process is then iterated until a stop criterion is met.

EAs have several characteristics that can help position themselves within the families of generation and testing methods:

- They are population-based, this means that they process a variable number of candidate solutions simultaneously.
- They usually use a recombination to inherit the information of more candidates in new solutions.
- These algorithms are stochastic.

The main components of an EA are listed below:

- Representation (definition of solutions).
- Evaluation function.
- Population.
- Parent selection.
- Variation, recombination and mutation operations.
- Survivor selection mechanism.

The above mentioned components are defined to a particular problem. When solving a problem, EAs usually perform the following steps:

- **Initialisation**: A population of individuals (solutions) is generated at random.
- **Evaluation**: Individuals are evaluated using the fitness functions.
- **Termination**: A criterion to determine whether the algorithm should stop or not.
- **Selection**: Select the solutions that survive to the next generation.
- **Variation**: This step can combines the information of two or more individuals to produce a new offspring solutions through a crossover operator. Mutation operation, conversely, changes the genetic information for a single individual.

.

2.3 Single-Objective Optimisation Algorithms

This section describes the different single-objective optimisation algorithms (SOAs) that will be used for the comparison.

2.3.1 Local Search

Local Search (LS), also referred as Hill Climbing in maximisation problems and Steepest Descent in minimisation problems, is a SOA based on the idea of generating random starting solutions. Then, at each generation, LS generates the starting solution for the next iteration by perturbing the local optimum found thus far.

It is worthy to mention that the perturbing mechanism is an important feature of LS. A weak perturbation may not be enough to escape the local optimum. Conversely, a too strong perturbation may make the algorithm too similar to a multi-start LS with randomly generated starting solutions. A pseudo-code for this algorithm can be seen in Algorithm 1.

```
Choose, at random, an initial solution s in the search space;
Apply a search starting from s to get a solution s*;
repeat
    Perturb s* to get a solution p;
    Apply search starting from p to get a solution p*;
    if f(p*) > f(s*) then
        s* ← p*;
    else
        // Do nothing
    end
until the stopping criterion is satisfied;
```

Algorithm 1: Local Search Algorithm

2.3.2 Genetic Algorithms

Genetic Algorithms (GAs) were developed in the early 70s in Michigan by J. Holland and his students, who were researching adaptive systems at the time [23]. The definition can be very generic and most of their parts are usually implemented differently according to each problem. Their essential parts include a representation of the solution, a selection strategy, evolutionary operators, and a mutation operator.

In GAs, the binary string is the most common representation. For a binary representation, the mutation is performed by doing a bit-flip in a gene. Conversely, the crossover operator recombines the genes of two or more individuals, by exchanging half of the information from one parent and a half from the other one.

Finally, the replacement or survivor selection uses the fitness value to identify the individuals for the next generations and is responsible to assure the survival of the fittest individuals. An example of an implementation of a GA can be seen in Algorithm 2.

```
Generate Initial population p;
Compute fitness of each individual;

repeat
    // Produce a new generation;
    for size of p / 2 do
        Select two individuals from the previous generation for matching;
        Recombine the two individuals to give two new off springs;
        Insert the offspring in the new generation;
    end
until stopping condition has been met;
```

Algorithm 2: Genetic Algorithm

2.3.3 Evolution Strategies

Similar to the GAs, Evolution Strategies (ES) follow the principles of natural evolution as their method to solve optimisation problems. They were introduced during the 70s by Rechenberg [27] and further developed by Schwefel [5]. The first algorithm used in experimentation was two-membered ES and had a simple mutation-selection scheme.

The basic structure consists of a single parent which produces an offspring following a normally distributed mutation resulting λ children. Next, a selection operator determines the fittest individual which then becomes the parent of the next generation.

Later, Rechenberg proposed a multi-member ES [27], where more than one parent can participate in the generation of one offspring individual. This is denoted as $(\mu + 1)$-ES where μ represents the number of parents. Two other definitions were then introduced by Schwefel [42], which were $(\mu + \lambda)$-ES and (μ, λ)-ES.

- $(\mu + \lambda)$-ES, known as elitist evolution strategy, indicates that μ parents will create $\mu \geq 1$ descendants by recombination and mutation, keeping the fittest individual of the previous generation and discarding the rest.
- (μ, λ)-ES, known as non-elitist evolution strategy, is similar with the difference that none of the parents is kept in any of the subsequent generations, disregarding how good or bad was its fitness compared to the generation they belong.

```
Initialise the population with random individuals;
Evaluate each individual repeat
    Select parents;
    Recombine pairs of parents;
    Mutate the resulting offspring;
    Evaluate new individuals;
    Select individuals for the next generation;
until the termination condition is satisfied;
```

Algorithm 3: Evolution Strategy Algorithm

2.3.4 Random search

Random Search (RS), also known as Pure RS, was first defined by Brooks [11] and later named in the classic volumes by Dixon and Szegö [26]. RS is considered as one of the most basic search strategies, and it consists of the following:

- It creates a new random solution
- The best solution is updated if the new solution outperforms the fit value of the previous best solution.

While this algorithm sacrifices the guarantee of determining the optimal solution within a given error, it can be shown that RS converges to the global optimum with a probability one [52]. It is commonly used to compare the performance of other search algorithms to random sampling [15].

2.3.5 Random Descent

Random Descent (RD), also known as Stochastic Hill climbing, is an algorithm similar to regular Hill Climbing. However, RD includes a neighbourhood, which limits the search to only the solutions that are close to the current best solution at each step. The most popular RD implementation was done by Forrest and Mitchell, naming it Random Mutation Hill Climbing (RMHC) [40].

The local improvements are determined by a neighbourhood structure, which can be seen as an undirected graph G on vertex set S, and a fitness function on the search space of the algorithm. The algorithm checks a member of its neighbourhood randomly and transitions to this solution when there is an improvement in the fitness function result or is the same. The process is then repeated at each step.Algorithm 4 describes the pseudocode for RD.

```
Currrent solution s from new population;
Evaluate the objective function f(s);
stop = FALSE;
iter = 0;
while stop = FALSE do
    Choose a random solution in the neighbourhood s_n;
    if f(s_n) > f(s) then
        replace s by s_n;
    else
        iter = iter + 1;
    end
    if iter = iter_max then
        stop = TRUE;
    end
end
```

Algorithm 4: Pseudocode for stochastic hill climbing

2.3.6 Parallel Tempering (Replica Exchange Monte Carlo)

Parallel Tempering (PT), also known as Replica Exchange Monte Carlo, consists in replicas of a system of interest were simulated at a series of different temperatures. Then, these replicas undergo a partial exchange of configuration information. The more general form of PT with a complete exchange of configuration information was introduced in 1991 by Geyer [19].

This algorithm runs several replicas of a particular system of interest, using different temperatures in each one of them. High temperature can generate large volumes of phase spaces, while low-temperature systems tend to have more precision in the sampling of the phase space and tend to be trapped in a local optimum.

Using this algorithm requires a minimum and maximum temperature, with each replica of the system with an assigned temperature. After each step, the replicas are ordered according to their temperature, and then swapped to push the better solutions to the lowest temperatures, so they can eventually converge while higher temperature replicas are constantly modified in a search for improvement. Each of the replicas applies repeatedly a series of moves to a private solution, and then the global solution is tracked in the main search. The pseudocode for this algorithm can be seen in Algorithm 5.

```
for i ← 0 to M − 1 do
    T_i ← calculate temperature i;
    replica_i ← new instance of Metropolis search with temperature T_i
end
repeat
    for i ← 0 to M − 1 do
        Run replica_i for N_swap iterations
    end
    G ← min(Cost(global_best_replica(0)), ..., Cost(global_best_replica(M−1)));
    j ← U{0, M − 1};
    // Randomly select adjacent temp. levels
    ΔE ← (Cost(active_solution_replica(j)) − Cost(active_solution_replica(j+1)))/G;
    Δβ ← 1/T_j − 1/T_{j+1};
    if U(0, 1) < min(1, exp(ΔEΔβ)) then
        Set the temp. of replica_j and replica_{j+1} to T_{j+1} and T_j, respectively;
        swap(replica_j, replica_{j+1}) // Keeps replicas ordered by temp.
    end
until The stop criterion has been met;
```

Algorithm 5: Parallel Tempering

2.4 Multi-Objective Algorithms

This section describes the different Multi-Objective Optimisation Algorithms (MOAs) that will be used for the comparison.

2.4.1 Multi-objective Random Search

Multi-objective RS is a probabilistic algorithm that serves mostly as an algorithm to benchmark other optimisation algorithms in the literature [55]. Just like its single objective counterpart, this produces a random set of solutions in each step. The output of the algorithm is the Pareto-optimal set of all solutions generated.

2.4.2 Electrostatic Potential Energy Evolutionary Algorithm

Electrostatic Potential Energy Evolutionary Algorithm (ESPEA) [47] is an algorithm that generates several Pareto front approximations, focusing only on the solutions that appear interesting to the decision-maker. Proposed in [47] after noticing that many of the algorithms found in the literature make use of popular crowding distance metrics that do not necessarily converge into optimality [16]. The algorithm attempts to design an electromagnetism inspired heuristic, where each solution has an assigned charge based on how close it is to a randomly selected subset from an archive of non-dominates solution. Then, the charges are translated to force vectors, which move the solutions in the search space. The best solutions are stored in an archive that uses clustering and crowding distance to prune the solutions.

```
Generate initial population P;
Copy all non-dominated solutions in P to archive A;
repeat
    Generate a single new solution p;
    Remove all solutions from A dominated by p;
    if p ≯ a ∧ f(p) ≠ f(a) ∀a ∈ A then
        if |A| < N then
            A := A ∪ {p};
        end
        else
            Calculate e(a) for all a ∈ 𝒜;
            Calculate e := (e_{a^1}(p), ..., e_{a^N}(p));
            update(A, p, e);
        end
    end
until stop criterion has met;
return A;
```

Algorithm 6: ESPEA

2.4.3 Many Objective Metaheuristic Based on the $R2$ indicator

Many Objective Metaheuristic Based on the $R2$ indicator (MOMBI2), introduced first by Hernandez Gomez and Coello Coello [25]. It has an advantage over other many-objective optimisation algorithms found in the literature, as a lot of them relies on non-dominated sorting and crowding distance, which can become inefficient as the number of objectives grows.

The $R2$ indicator was introduced by Hansen and Jaszkiewicz [48]. The $R2$ indicator compares two sets of points being a reference Pareto front and the approximation of the front. MOMBI2 uses $R2$ because it induces the complete ranking in the set of all approximations. It is based on the assumption that it is allowed to add values of different utility functions in the set. Therefore, it is also dependent on the scaling of its utility functions. Algorithm 7 describes MOMBI2.

Require: MOP, stopping criterion,set of weight vectors W;
Ensure: Pareto set approximation;
Initialise population $P_i, i \leftarrow 1$;
Evaluate population $P - i$;
Calculate the $L_2 - norm$ of objectives for P_i;
Set $\vec{z}^{min} \leftarrow \vec{z}^{*}$ and $\vec{z}^{max} \leftarrow \vec{z}^{nad}$;
while *termination condition is not fulfilled* **do**
  Perform parent selection;
  Generate offspring P_i' using variation operators;
  Evaluate population P_i';
  Calculate the $L_2 - norm$ of objectives for P_i';
  Normalize objective functions for $P_i \cup P_i'$;
  Execute $R2$ ranking algorithm $(P_i \cup P_i', W)$;
  Reduce population $P_{i+1} \leftarrow \{P_i \cup P_i'\}$;
  Update reference points $(\vec{z}^{min}, \vec{z}^{max}, P_{i+1}, m)$;
  $i \leftarrow i + 1$
end
return P_i

Algorithm 7: MOMBI2

2.4.4 Non-dominated Sorting Genetic Algorithm II

Non-dominated Sorting Genetic Algorithm II (NSGA-II), proposed by Deb et al. [16], makes use of the Crowding Distance which is a certain preference for solutions that are less crowded in the solution space.

This algorithm works as follows. After an initial population is created, each individual is ranked and sorted according to their non-dominance level, each solution then is assigned a fitness according to their non-domination level (1 is assigned to the best, then 2 and so on). Then a binary tournament selection, crossover and mutation take place and an offspring population of equal size is created. The process is then repeated joining the previous generation and their offspring as the same population, and it continues with the population always taking the parents of the previous generation; therefore, elitism is ensured. It is important to notice that in the following generations, the ranking will also consider the crowding distance. The pseudocode for this algorithm can be seen in Algorithm 8.

```
Generate a random initial population P_t;
R_t ← P_t ∪ Q_t ;
// combine parent and offspring population
;
F ← fast-non-dominated-sort(R_t);
// run the fast non dominates sort procedure and collect
   F = (F_1, F_2, ...) with the non-dominated fronts of R_t
;
P_{t+1} = 0 and i = 1;
repeat
    crowding-distance-assigment(F_i);
    // calculate crowding-distance in F_i
    ;
    P_{t+1} = P_{t+1} ∪ F_i;
    i = i + 1;
until |P_{t+1}| + |F_i| ≤ N ;
// until the parent population is filled
;
Sort(F_i, ≺_n);
P_{t+1} = P_{t+1} ∪ F_i[1 : (N − |P_{t+1}|)];
Q_{t+1} = make-new-pop(P_{t+1});
t = t + 1;
```

Algorithm 8: NSGAII

2.4.5 Strength Pareto Evolutionary Algorithm 2

Strength Pareto Evolutionary Algorithm 2 (SPEA2) [54], first introduced by Zitzler and Thiele in 1999, was among the first techniques that were extensively compared to several existing evolution-based methods [12].

SPEA2 makes use of an external non dominated set. For each individual in this set, a strength value is computed, which consist of the raw fitness value assigned from the objective functions and a density estimation similar to the ranking value of MOGA [22]. The algorithm applies the respective selection, crossover and mutation operators to fill an archive of individuals, then applies this fitness function and the non-dominated individuals from both the original population and the archive are copied into a new population. If the number of non-dominated individuals is greater than the population size, a truncation operator based on the distance to the k-nearest neighbour is used. Hence, the individuals having the minimum distance to every other are discarded. Pareto dominance is used to ensure that the solutions are properly distributed along the Pareto front. SPEA2 differs from its predecessor SPEA in three ways:

1. **Considering the fitness assignment**, the individuals that are dominated by the same archive members have identical fitness values. That means, in case that the archive

has only one individual all population members will have an independent rank value, therefore lowering the selection pressure and making SPEA behave like a random search algorithm.

2. **For the density estimation**, if many individuals are indifferent to the current generation, which means there is no domination between them, there is little to no information that can be obtained from them, which happens rather frequently, density information is used to guide the search more effectively. Clustering makes use of this information but only regarding the archive and not the population.

3. **Archive Truncation**, considers that, while in SPEA the clustering technique was able to reduce the non-dominated set, without destroying its features, it could lose outer solutions, however, these solutions must be kept in the archive to keep the spread of non-dominated solutions.

Then the fitness values are assigned to the archive and the population members.

- Each individual i is assigned a strength value $S(i) \in [0, 1)$, which at the same time considers the fitness function $F(i)$. $S(i)$ is the number of population members j that are dominated or equal to i, divided by the population plus one.

- The fitness $F(j)$ of an individual j in the population is calculated by summing the values of $S(i)$ of the rest of the archive members that dominate or are equal to j plus one.

Then for the mating selection, individuals are selected by binary tournaments. This assumes a minimisation, each member of the archive has a higher chance of being selected than the rest of the population. Finally, after crossover and mutation operators are applied, the population is replaced by the resulting offspring population. The pseudocode for this algorithm can be seen in Algorithm 9.

Input: *population size* N, *archive size* $\bar{N}$, *maximum number of generations* T;
Output: *nondominated set* **A**;
Initialize: Generate an initial population P_0 and create the empty archive $\bar{P}_0 = \emptyset$. Set $t = 0$;
repeat
  Fitness assignment: Calculate fitness values of individuals in P_t and $\bar{P}_t$;
  Environmental selection Copy all nondominated individuals in P_t and $\bar{P}_t$ to $\bar{P}_{t+1}$. ;
  if *size*$(\bar{P}_{t+1}) > \bar{N}$ **then**
    Reduce $\bar{P}_{t+1}$ by means of the truncator operator
  end
  else
    Fill $\bar{P}_{t+1}$ with dominated individuals in P_t and $\bar{P}_t$
  end
until $t \leq T$ *or any stopping criterion is satisfied.*;
Termination: **A** $\leftarrow$ the set of decision vectors represented by the nondominated individuals in $\bar{P}_{t+1}$;
Mating selection: Perform binary tournament selection with replacement on $\bar{P}_{t+1}$ in order to fill the mating pool.

Algorithm 9: SPEA2

2.5 Comparing Single-Objective and Multi-Objective Optimisation

H. Ishibuchi et al. [30] presented a general study of this topic. The experiments are related to the comparison of single-objective GAs (SOGA) and multi-objective GAs (MOGA). In this work is also discussed the several difficulties occurring when comparing both types of algorithms, such as the inability to use hypervolume (HV) as a comparison metric, since it strongly depends on the location of the origin.

Different works include a comparison between single-objective and multi-objective strategies for different fields of the industry [39, 34, 45]. For example, Jiao and Zeng [32] among others make use of different strategies to convert a mono-objective problem into a dynamic multi-objective one in which apply a multi-objective evolutionary algorithm with improved results compared to existing solutions. Another recurring theme in which their performance is compared is in the design of antennas to improve cellular reception, as is the case of the work of Rahmat-Samil and Nanbo Jin. [33].

Finally, Battiti and Passerini [6] talk about the use of GAs that adapt to the decision-maker, this is relevant because in the final platform the user does not properly have a real decision on which group will end up being assigned but this is intended to be done automatically. Battiti mentions some of the possible strategies for adapting the behaviour of a decision-maker and also discusses some of the limitations that such a system would present according to their preferences [51].

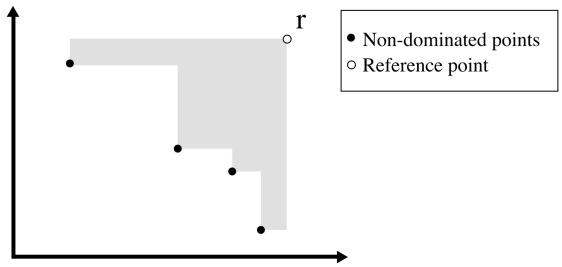


Figure 2.1: Demonstration of the (HV) in two dimensions, the grey area represents the value of $HV(S, r)$

2.6 Metrics

The next section includes metrics to use for comparison with multi-objective algorithms. The only metric that has shown to represent the domination of a set to another is the HV, however this metric is considered costly as the number of objectives and solutions grows and is usually considered to compare the performance between different algorithms, while the rest of the metrics serve as good approximations of the HV, that require less computational cost and can be calculated in each generation or step to choose a set of solutions over another. Besides there exist other sets of metrics such as spread that measure the distribution of the solutions within a given front, this helps determine the quality of the front because it considers a wider space covered within the search space.

2.6.1 Hypervolume

First defined by Zitzler and Thiele [56], as:

Definition 5. *The size of space covered S by a front $X' = (x_1, x_2, ..., x_k)$, as a set of points in a Front, then $S(X')$ gives the volume enclosed by the union of the polytopes $p_1, p_2, ...p_k$ where each p_i is formed by the intersection of the hyperplanes arising out of x_i along with the axes, for each of them in the objective space, there exists a hyperplane perpendicular to this axis and passing thought the point $(f_i(x_i), f_2(x_i), f_n(x_i))$. In the two dimensional case, each p_i represents a rectangle defined by the points $(0, 0)$ and $(f_1(x_i), f_2(x_i))$.*

HV is considered the only unary indicator that is capable of truly detecting that a front A is not worse than another front B for all its pairs $A \triangleright B$ [57]. This concept is referred as *pareto complaint* [28]. It is also noticed that for all the points in a front A and a front B, $A \triangleright B$ follows $I_H(A) > IH(B)$, because this means that A must include at least one point that is not weakly dominated by B, therefore a portion of the objective space is covered by A but not by B.

This can be more clearly seen in Figure 2.1, wherein the former the grey area represents the $HV(S, r)$ in the search space.

One more general definition can be found in [3] where it is described as the volume of the space in the objective space dominated by an approximation of the Pareto Front S and delimited from the above by a reference point $r \in R^m$, so, for all the points z in the obtained front. The hyper-volume is given by:

$$HV(S, r) = \lambda_m \left(\bigcup_{z \in S} [z; r] \right) \tag{2.5}$$

Where λ_m is the $m-$dimensional Lebesgue measure.

One of the major drawbacks of using HV is that its complexity results in $\mathcal{O}(|S|^{m/2} log|S|)$ and is also the main reasons other measurements are preferred over HV especially in many-objective problems. It has been used without issues with the number of objectives lower than 10 and a non-dominated number of solutions less than 10000. However other methods and algorithms have been proposed to address this issue [3].

2.6.2 Spread

First defined by Deb et al. in [57], the metric spread often represented as $\triangle$. It measures the extent of spread achieved among the obtained solutions. Its goal is to get a set of solutions that spans the entire Pareto-optimal region.

The euclidean distance d_i is calculated between each consecutive solution in and obtained front. Then the average $\overline{d}$ is calculated for these distances, so for a front composed of non-dominated solutions, first the extreme solutions of the objective space are obtained, these are considered the boundaries of the obtained solutions, this is done by fitting a curve parallel to the Pareto front. Afterwards, the spread is calculated using Equation 2.6.

$$\Delta = \frac{d_f + d_l + \sum_{i=1}^{|NDS|-1} \left| d_i - \bar{d} \right|}{d_f + d_l + (|NDS| - 1)\bar{d}} \tag{2.6}$$

Here d_f and d_l are the Euclidean distances between the extreme solutions, this can be better seen in Figure 2.2 where all the non-dominated solutions are found within the curve drawn.

There may exist a scenario where are solutions lay within the same point, this is not considered the worst case of spread. When there is a great variance between the distance of the solutions the increment can be greater than one. On the other hand, a good distribution would make all the distances d_i equal to the average $\overline{d}$, making d_f and d_t equal to zero, making $\triangle$ also zero. In the case of higher dimensions, a triangularization technique is used to calculate the distances for the solutions.

This metric was one of the most preferred in the literature. However, its usage has decreased in the recent years. One of the major drawbacks is that the metric alone only measures the diversity of the approximation to the Pareto front [43]. Nevertheless, combined with other convergence metric is still a relevant approach to assess the quality at a low computational

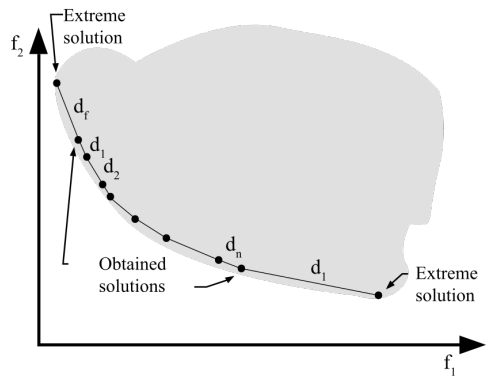


Figure 2.2: Graph representing the Spread metric.

cost. It should also be noted that it remains as the most popular metric to measure the diversity, but there is no more interest in the literature to measure the diversity alone to determine a quality set of solutions.

2.6.3 Epsilon

First defined in by Zitzler and Thiele in 2003 [57]. For a long time, it was considered the main indicator to describe the domination of the solutions [43]. It consists in getting a factor ϵ from which if an objective front are multiplied the Pareto front or any another front still dominates the former.

In the case of minimisation, a vector $\vec{z_1}$ is said to $\epsilon-$dominate another vector $\vec{z_2}$ if and only if for all the points of the vector $\vec{z_2}$ multiplied by a factor of ϵ remains higher than $\vec{z_1}$, this can be best described in Equation 2.7.

$$I_{\in}(A, B) = \inf_{\epsilon \in R} x\{\forall \vec{z_2} \in B \exists \vec{z_1} \in A : \vec{z_1} \succeq_{\in} \vec{z_3}\} \tag{2.7}$$

Loosely speaking, a vector $\boldsymbol{z}$ is said to epsilon dominate another vector if the multiplication of each objective value in the other vector by a factor of e and the resulting objective vector is still weakly dominated by the first. The resulting Epsilon is the E precise factor.

According to this the ϵ-indicator gives the factor by which an approximation set is worse than another with respect to all its objectives, therefore $I_{\in}(A, B)$ equals to that minimum factor ϵ such that any objective in B multiplied by the factor ϵ is ϵ-dominated by at least one objective vector in A. For a single objective evaluation $I_{\in}(A, B)$ is simply the ratio between the two objectives A and B.

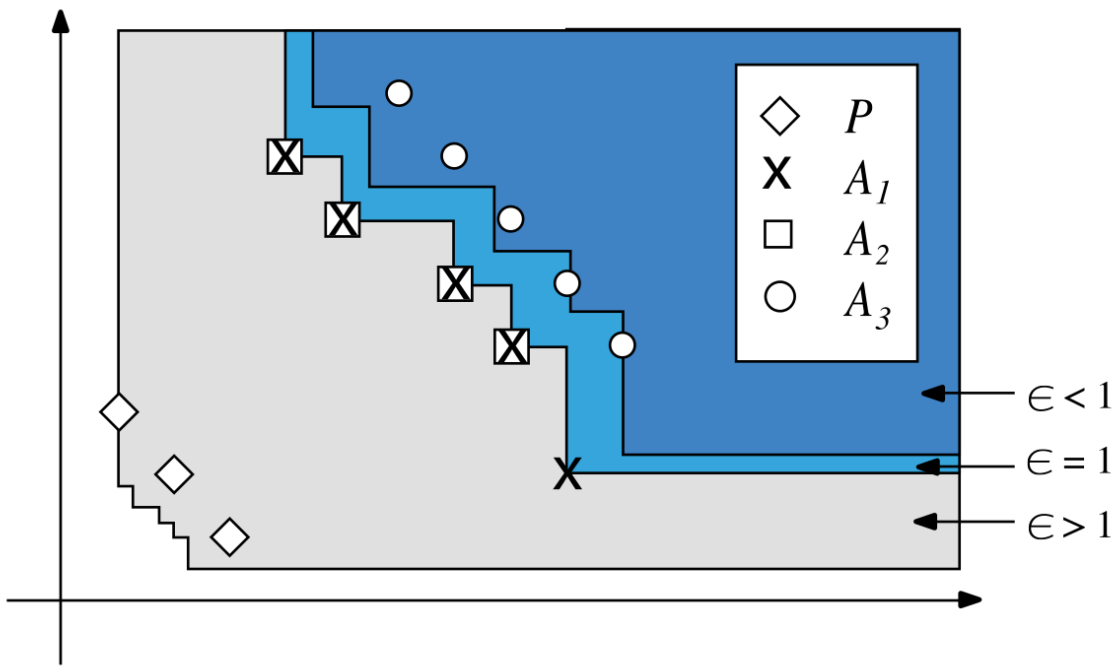


Figure 2.3: Graph representing the subspace of which Epsilon is based.

For example, in Figure 2.3, the dark blue area represents the subspace ϵ-dominated by the solutions in A_1 for an $\epsilon = 9/10$, the light blue area refers to the subspace ϵ-dominated by the solutions in A_1 for an an $\epsilon = 4$.

2.6.4 Generational Distance

Generational Distance (GD), was first used by Van Veldhuizen and Lamont in the early experiments done in 1998 [49]. GD is a value that consists in representing how far a known Pareto front is from the real Pareto front defined in Equation 2.8.

$$G \triangleq \frac{\left(\sum_{i=1}^{n} d_i^p\right)^{1/p}}{n} \tag{2.8}$$

where n is the number of solutions in the known Pareto front and d_i is the Euclidean distance in the objective space between each of the solutions and the nearest member of the real Pareto front, and p representing the number of objectives. When the known and the real Pareto front are the same the distance should be zero. Any other result shows how much the known Pareto Front deviates from the real one.

GD has been lowering its presence in the literature in recent years, as HV has been increasing is usage [43]. One of its main drawbacks in comparison to other metrics is that if a known front presents a large fluctuation in the distance values the calculated value may not represent the true distance.

2.6.5 Inverted Generational Distance

Inverted GD (IGD) [13] can be calculated as the inverse of the GD starting with the Real Pareto Front and calculating the distance to the fronts produced by their tested algorithms [28]. It can also be calculated by the following equation:

$$D\left(A, P^{*}\right)=\frac{\sum_{v \in P^{*}} d(v, A)}{\left|P^{*}\right|} \tag{2.9}$$

Where $d(z_i, a_j)$ is the distance be a distance between z_i and a_j in the objective space. It is mentioned by [53], that If $|Z|$ is large enough to represent the Pareto Front well, then $D(A, P*)$ can be used to measure both the diversity and the convergence.

There are two main advantages of using IGD, the first is its computational efficiency even considering multi-objective problems with more than four objectives, called many-objective problems, especially comparing it to HV. The other is its generality, it usually shows the overall quality of an obtained front, considering its convergence and to it and its diversity. This is the main reason why its often used in the found literature [29].

However, besides the advantages gained from IGD it still has several disadvantages which are discussed in the following subsection.

2.6.6 Inverted Generational Distance +

While IGD is increasing its use, there are several disadvantages mentioned in [28], one of these known flaws, especially comparing it to the HV is its lack of Pareto compliant property, this can be further be seen in Figure 2.4

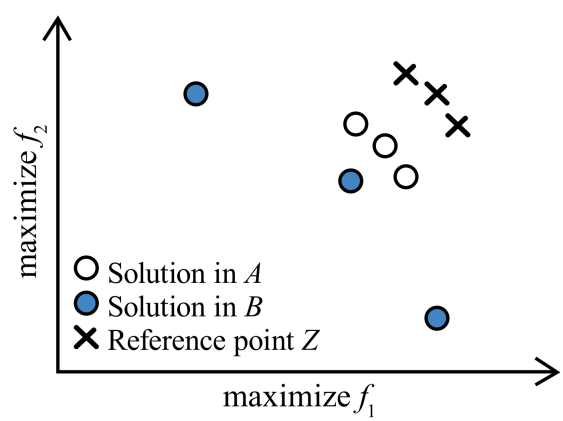


Figure 2.4: Example of distance between Reference Z, Solution A and Solution B.

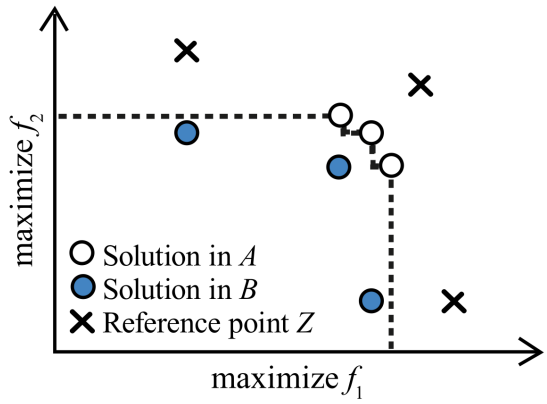


Figure 2.5: Example of distance between Reference Z, Solution A and Solution B, here the solutions from set B appear to be closer to Z.

This presents a comparison between the front $A = (a_1, a_2, a_3)$ and the front $B = (b_1, b_2, b_3)$, and a reference Pareto front $Z = (z_1, z_2, z_3)$ for a maximisation problem. Here every solution in the front B is clearly dominated by the solutions in the front A, however the measurement of IGD between Z and B would be lower, because of the way the solutions in B are distributed which has a similar spread to Z.

To address these drawbacks, IGD+ was proposed by Ishibuchi and Masuda [28]. It is closely similar to IGD with the difference of having a weak Pareto compliant property, and it is based on the idea of calculating the distance from each reference point to a dominated region instead of the nearest solution, this can be seen in Figure 2.6.

Then, $(IGD+)$ is defined as the distance between a Pareto front Z and a solution front A.

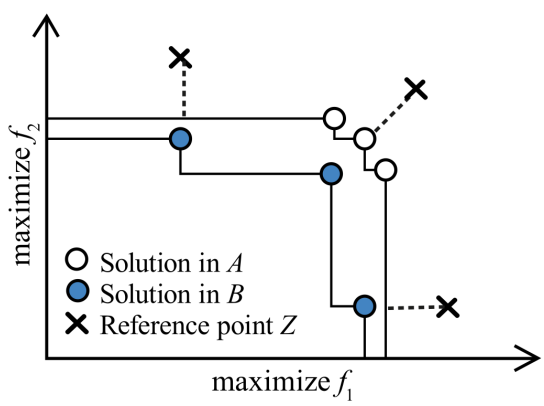


Figure 2.6: A graph representing the main idea behind $(IGD+)$.

Here, if a solution A is dominated by the reference front Z, then $d_{IGD+}(A, Z)$ is the same as the euclidean distance $d(A, Z)$, because all the solutions in the front A are smaller

than those from the reference front Z, and this also holds for each point in their respective fronts. When A and Z are not dominated with each other, only the objectives in A that are lower to their counterparts in Z are used, which means that if a point in the front A is better than another from point Z with respect to some of its objective, then these are not used in the calculation of IGD+.

2.7 Frameworks

2.7.1 jMetal

jMetal is a framework based on the Java programming language. It is focused on development, experimentation and study of meta-heuristics to solve MOPs. JMetal includes various of the most common algorithms. A series of example problems and regularly used indicators to measure the performance of algorithms. It also includes tools to carry out experimental studies that can be configured to generate statistical reports of the obtained results. It is also possible to take advantage of a multi-core architecture to make the experimentation process faster [18].

Some of the features sought are part of this type of software are:

- Include the most current algorithms available
- Contain the most accepted performance tests for multi-objective problem solving
- Give quality indicators to measure the performance of each of the tests and assist in a more accessible way in the investigations of its users.

2.7.2 James

James, which stands for Java Metaheuristics Search, is an object-oriented Java framework for discrete optimisation using local search algorithms that takes advantage of the generality of such meta-heuristics by separating search implementation and application from each problem specification. A wide range of generic local searches is provided, including: (Stochastic) Hill climbing, Tabu search, Variable neighbourhood Search and Parallel Tempering. These can be applied to any user-defined problem by designing a custom neighbourhood for the corresponding solution type. Using an automated analysis workflow, the performance of different search algorithms can be compared to select an appropriate optimisation strategy [15].

Implementations of specific components are included for subset selection, such as a predefined solution type, generic problem definition and several subset neighbourhoods used to modify the set of selected items. In comparison with existing Java meta-heuristics frameworks that mainly focus on population-based algorithms, JAMES has a much lower memory footprint and promotes the efficient application of local searches by taking full advantage of move-based evaluation and parallelization.

2.8 Preference Criteria

This section describes the different metrics used as a basis for the objective functions based on the predicted preference of the students according to the group they want to belong.

2.8.1 Benne and Sheats functional roles

The Benne and Sheats functional roles are a tool that helps define what kind of reaction a person has at the time of starting a conversation [8]. The roles we are specifically interested in are known as the **Maintenance Roles**.

- **Attacker**: Is that person who invalidates the comments of others and ”attacks” the rest of the group members.
- **Protagonist**: is the person who initiates the conversations, assumes an authority role. In this case, it is a very valuable role in our research.
- **Follower (or Supporter)**: This person has a cooperative attitude, pays attention to the talk, accepts the proposals and offers technical support.
- **Gatekeeper**: is a person who plays the role of moderator, checking that each person has their opportunity to participate similarly.
- **Neutral**: It is similar to the follower role, only instead of actively participating, it accepts the comments passively.

It should be noted that a person can represent different types of roles in the same conversation, although it is generally classified depending on which was their most prominent role.

AMI Meeting Corpus

AMI meeting corpus is a data-set containing the recording of several conversations in different contexts. With the purpose to analyse the interaction between the participants. It has been used in several works to predict participation styles such as Vinciarelli [50] who uses this database to identify the functional roles of Benne and Sheats [8].

2.8.2 Facebook Ad Platform

The social network known as Facebook is useful to maintain contact between friends and acquaintances, however, it is also a useful marketing tool to give users who belong to certain market ads focused on each of them according to their preferences, behaviours and locations. This follows a marketing methodology known as Demography, Behaviours and Interests. As far as this research is concerned, we are only interested in the Interests.

These interests are determined by the classification of the pages or groups that the user follows, however it has a more general classification as an ontology that Facebook has been

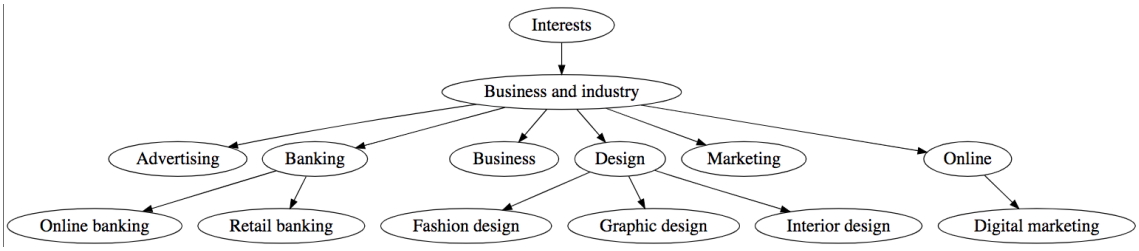


Figure 2.7: An example of the Facebook interests ontology.

building over the years using different clustering techniques so that It can have a more general categorisation of the different types of interests of each of the users.

Besides, Facebook allows businesses through a platform known as **Facebook ad Platform** [1] publish different types of ads aimed at a type of market according to the interests or behaviours corresponding to a market. This platform is public and can be used to determine an approximate cost when preparing marketing campaigns. However, this research is used to determine the population size for each of the interests described as it will be mentioned in Sections 3 and 4.

This platform can also be used at the same time to generate the distribution of the data since it is directly linked to the ontology and can be consulted to measure the distances between one interest and another. Part of the ontology can be seen in Figure 2.7.

The ontology that Facebook has created is determined with a tree of 8 general and 314 specific interests. Each classified according to the relation to the general interest. This allows to search for specific interests in common, for example, if a person likes television programs we can also assume that she may also like reality shows or contest programs etc. In this way, another person who has an interest in television series has a direct relationship with the first. On the other hand, interests that have no relation, have no connection in the ontology.

2.8.3 CEFR

Known as **Common European Framework of Reference for Languages** for its acronym in English, it is a tool used internationally to assess the level of knowledge of a foreign language and is commonly known throughout the world in such a way that although a different test has been performed, it is possible to find a type of equivalence using this classification, which allows being able to take into account at the level of a person regardless of what type of test taken.

The levels are classified as A1, A2, B1, B2, C1 and C2. The first two levels, A1 and A2, refer to the most basic levels where a person knows the basics of the language, such as introduce themselves or asking for directions. The second tier, B1 and B2 are considered the intermediate level, people in this classification can understand long casual conversations as

well as writing and reading non-specialised content. The last two levels C1 and C2 refer to people with the ability to deeply understand and speak the language, moreover, to write full documents using it.

Chapter 3

Methodology

This chapter explains the different parts of the problem and the strategies to solve it. Section 3.1 describes how the problem is structured, the size of the search space, and how a feasible solution looks like. Then, Section 3.2 describes the objective functions, as well as a justification of why each of them was chosen. Finally, Section 3.4 defines the operators to be used by the optimisation algorithms and the reasoning behind them.

3.1 Problem Design

The problem established is described as given a data set D of n students, $D = (s_1, \ldots, s_n)$, a solution A consists of an optimal arrangement of groups G, which assigns each student to a group, $A = \langle s_i \leftarrow G_j, \ldots, s_n \leftarrow G_m \rangle$, with the students being in the same group optimising their preference criteria as described in Section 2.8. Thus, the optimisation encompasses the following goals:

- Finding the less distance between the level of experience of the students inside the same group.
- Finding the less distance among an ontology of interests, where similar interests related interests are in the same branch.
- Having a group size near the ideal group size which is arbitrarily defined as between four and five.
- The percentage of participation which is how much the students participate in the conversation of the group, maximising for the most participation.

The solution is then the group number for each student, which serves as identification. The solution should ensure maintaining the preference of the user within the group, regardless of the order in which the groups are presented. This can be seen in Figure 3.1 and Equation (3.1).

$$\begin{gathered} D = (s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_9, \ldots, s_n), \\ A = \langle s_0 \leftarrow G_3, s_1 \leftarrow G_0, s_2 \leftarrow G_2, s_3 \leftarrow G_1, s_4 \leftarrow G_3, s_5 \leftarrow G_1, \ldots \rangle \end{gathered} \tag{3.1}$$

There are two manners to approach the problem. The first one considers the groups as the variables and the students as the values, while the second one considers the students as the variables and the groups as the values. The second approach, however, has the disadvantage that the number of variables becomes inconsistent depending on the size of the groups. Another disadvantage is that some groups could remain empty, and the crossover and mutation added unnecessary complexity to the task. The former approach does not present these disadvantages and is the chosen one to use.

A linked list is further kept to internally track the groups and make easy the evaluation of objective functions, which means if $s_i \leftarrow G_j$ then $G_j = (\dots, s_i, \dots)$.

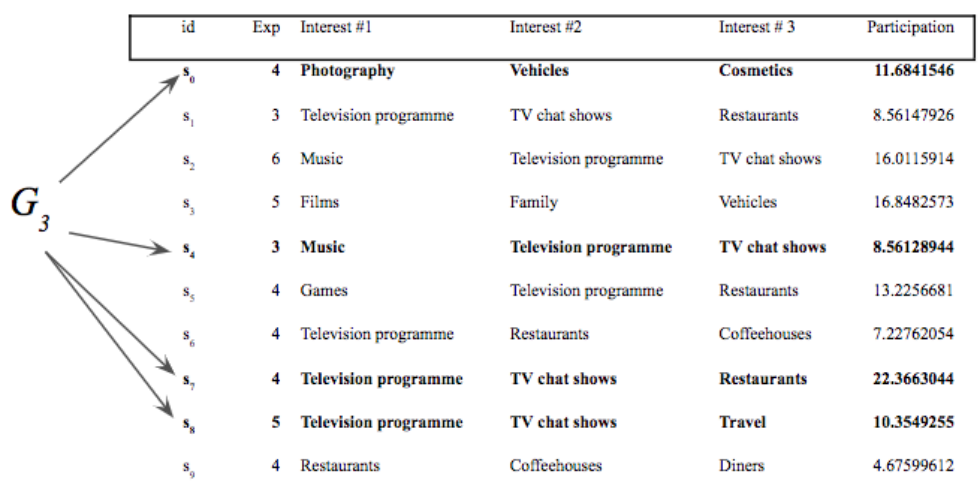
The search space comprehends all the possible assignments of groups to the students. This is related to the Bell number, which counts the possible partitions of a set. The Bell number is defined recursively as:

$$B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k \tag{3.2}$$

where B_k is the number of partitions for a set with k elements.

The neighbourhood is composed of every solution that has a single student in a different group, among the feasible solutions.

Figure 3.1 presents a sample of a dataset with each of its different features, as it can be seen the Group G_3 is assigned to students according to the preference criteria.



id	Exp	Interest #1	Interest #2	Interest # 3	Participation
s_0	**4**	**Photography**	**Vehicles**	**Cosmetics**	**11.6841546**
s_1	3	Television programme	TV chat shows	Restaurants	8.56147926
s_2	6	Music	Television programme	TV chat shows	16.0115914
s_3	5	Films	Family	Vehicles	16.8482573
s_4	**3**	**Music**	**Television programme**	**TV chat shows**	**8.56128944**
s_5	4	Games	Television programme	Restaurants	13.2256681
s_6	4	Television programme	Restaurants	Coffeehouses	7.22762054
s_7	**4**	**Television programme**	**TV chat shows**	**Restaurants**	**22.3663044**
s_8	**5**	**Television programme**	**TV chat shows**	**Travel**	**10.3549255**
s_9	4	Restaurants	Coffeehouses	Diners	4.67599612

Figure 3.1: This is an example of a data set of nine users, each with their experience, three different interests and a participation percentage.

3.2 Objective Functions

In the following section, the different objective functions are described, which are based on the preference criteria defined in Section 2.8, the experience level, the interests similarity, and the participation style. It should be noted that preference criteria are evaluated per group G.

Then, the objective values for a solution A are computed as the average of each preference criterion. For instance, for the Group Size $Gs(A) = AVG(Gs(G_i), \ldots, Gs(G_n))$.

3.2.1 Group Size

For this research, the ideal group size Gs is arbitrarily considered from four to five students. This size is based on the idea that it will allow users belonging to the same group to participate homogeneously. It is also possible that there might be groups with $gs_{min} = 3$ or $gs_{max} = 6$ students, but these are not considered as ideal group sizes. Nonetheless, these non-ideal groups sizes are still considered as feasible group sizes, so that other features, such as the *participation style* or the *interests similarity* results in a better cohesion of the group.

The corresponding objective function measures the distance to 4.5 since four and five are considered equally ideal and 4.5 is just the average between both. Thus, the objective function is defined as:

$$Gs(G_i) = |Gs(G_i) - 4.5| \tag{3.3}$$

3.2.2 Experience level

The experience level Lvl in the language is taken from the standard **CEFR**. The values considered for each level are A1, A2, B1, B2, C1 y C2, which are sorted from the most basic to the most advanced level. Therefore, a numeric value is assigned to each of them from zero to five, where zero corresponds to A1 and five corresponds to C2. The intention for assembling the groups is that the users inside them do not have much variation in their experience level. In this way, the vocabulary used is not too advanced for the lower student levels or too simple for more advanced users.

Equation (3.4) calculates the standard deviation σ of the level of each student is considered. An ideal group should have a standard deviation of zero, indicating that all users from a group have the same experience level.

$$Lvl(G) = \sqrt{\frac{1}{Gs(G) - 1} \sum_{i=1}^{Gs(G)} (Lvl(s_i) - \overline{Lvl(s)})^2} \tag{3.4}$$

where $Gs(A)$ is the size of the group, $Lvl(s_i)$ is the level of a particular user and $\overline{Lvl(s)}$ is the average of the level of all the users in the group.

3.2.3 Interests Similarity

To calculate the interests similarity Int, a strategy similar to the one described in[36] is used. All the interests are considered a vector of values, where each of them is ranked based on the distance it takes to reach each other interest in the ontology defined by the Social Network Facebook. Each interest has assigned a value of d_i, which indicates this distance. It represents how related is a particular interest related to others. For example, if the ontology is seen in Figure 3.2 is taken into account, the interest ”Graphic Design” is a branch of ”Design” which in turn is a branch of ”Business and Industry”, therefore it would have a value of two with

respect to ”Business and Industry”, a value of one with respect to ”Design” and a value of zero for itself.

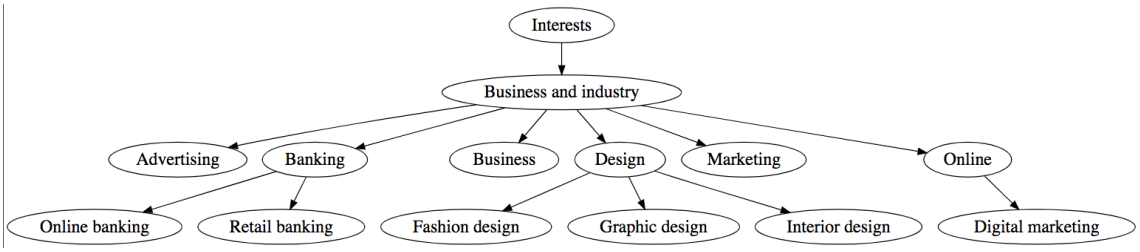


Figure 3.2: An example of the Facebook interests ontology.

These values are then normalised by a factor of $1/(1+d)$. The interest vector for a user is confirmed by the maximum value of this normalisation. For example, a student s_a, with ”Graphic Design”, ”Interior Design”, and ”Banking” as his/her interest, has an interest vector equals to $Ints(s_a) = \langle Business \leftarrow 0.50, Design \leftarrow 0.50, GraphicDesign \leftarrow 1.00, \ldots \rangle$ and a student s_b, with ”Retail Banking”, ”Advertising”, and ”Running” as his/her interest, has an interest vector equals to $Ints(s_b) = \langle Business \leftarrow 0.33, Advertising \leftarrow 0.50, Banking \leftarrow 0.50, \ldots \rangle$. The complete process can be seen in Table **??** for the student s_a and in Table **??** for the student s_b.

To compare the interests of a student to another student, the cosine $cos(\theta)$ distance, shown in Equation (3.5), is used. When the users have identical interests, their cosine distance becomes one, and zero for completely different interests. Since we are minimising, the objective function is considered as $f = 1 - f_{int}$. An example of two interests vectors of two users can be seen in Table **??**, here $1 - f_{int}(s_a, s_b) = 0.29$ which is considerably similar because both students share interests such as:

$$f_{int}(s_a, s_b) = \frac{\sum_{i=1}^{n_{int}} Int(s_a)Int(s_b)}{\sqrt{\sum_{i=1}^{n_{int}} Int(s_a)^2}\sqrt{\sum_{i=1}^{n_{int}} Int(s_b)^2}} \tag{3.5}$$

where $Int(s_a)$ is the interest vector of a user and $Int(s_b)$ is the interest vector of another and n_{int} is the number of Interest of s_a and s_b combined.

The similarity of interests of a group G is the average of all the interests function $f_{int}(s_a, s_b)$ for all the students in the group.

3.2.4 Participation style

There is no defined function that allows calculating participation of Ps in a group or a special configuration of participation styles, which promotes the participation of all students. However, as mentioned in Section 2.8.1, we make use of Benne and Sheats functional roles [9]. One particular role is defined as *protagonist*, which is a role that implies participation. A member in a group does not have a particular role defined, but they may have different roles as the conversation goes on. Therefore, a percentage of *protagonism* can be defined for each student representing how much was the role *protagonist* present in a conversation.

Since it is desired to have high participation in a group, and the problem is minimisation, then the participation style is considered as the inverse of the sum of all the participation percentage of protagonism of the students of a group, this is simply defined in Equation (3.6).

$$Ps(G_i) = \frac{1}{\sum P(s_i)} \tag{3.6}$$

Where $Ps(s_i)$ is the percentage of protagonism of each of the students of the group.

3.3 Stability

Stable Student Groups are described as:

Definition 6. *For a Solution, A is considered stable if and only if there is no student from any group G_a that would prefer to belong to another group G_b, i.e., students are satisfied with the group that they belong to.*

The reason a student s_a may want to leave their group for another is that they perceive a considerable gain from doing the switch. For example, the students from G_b might have more common interests with s_a. This preference is defined by the factors from the preference criteria, the interests similarity, the level similarity, their participation style compared to the other students from their group and the group size.

Taking this into account, the satisfaction according to their preference is calculated using the same objective functions defined earlier. Once the groups are made, the evaluation of each of the objective functions is assigned from each student towards the rest of the members of the group, and the other groups. ”How much they would like to belong to any other group?”

Suppose we have the following data-set D:

Let’s assume we find the solution: $A = \langle G_0 \leftarrow (s_0, s_1, s_2, s_4, s_5), G_1 \leftarrow (s_3, s_6, s_7, s_8, s_9)\rangle$, for this example we make use of the Interests Similarity function alone. In Table **??** the Interest Similarity is calculated from each user towards every other from the data-set. Then, the average of these values is calculated, this value represents ”How much they would like to leave their group for the other?” according to the rest of the members. After that, the same is done for the other groups, representing ”How much they would like to leave this group if they were a member?”

Here all the preferences for each one of the students are satisfied except for s_0, who would prefer to be in the group G_1 since the average of the similarity is lower (which means more similar) to the other group than the former. The overall stability of the solution is given by the number of stable groups, which means all of their students are satisfied with the arrangement. Here, G_1 is considered stable because there are no students that would like to switch to G_0. However, G_1 is considered unstable because there is at least one student that would prefer to belong to G_1.

Table **??** shows the case when s_0 is moved to G_0. There, we can appreciate that now s_5 is unsatisfied with their group, so the group G_1 is now considered unstable. There still an unsatisfied student and an unsatisfied group, so both solutions are considered equally stable.

Now, let us consider the rest of the preferences, using their respective objective functions. We evaluate each one of them in the same way, collecting the satisfied and unsatisfied students according to each preference objective function in the same way as above.

With multiple criteria, the interpretation of stability can vary, it can be said that the solution may be $Lvl-Stable$ or $Int-Stable$ for example. When considering multiple-preference criteria, general stability could have multiple interpretations. Once again, the purpose of this research is not to have perfectly stable solutions, but near-stable solutions that are considered good enough, as long as the stability inside the groups is higher than the groups from a random solution. Therefore, the way the stability is measured is by counting the number of students that are satisfied in each of their individual preferences as well as all their preferences combined.

For this example, the results are shown in Table **??**. Here the stability is analysed for each of the different preference criteria, it can be noticed that not all the students are satisfied with their groups according to $Ps - Stable$ and $Int - Stable$, but this result may still be perfectly-stable.

An Important thing to notice is that the preference satisfaction for the group size is considered met by all the students if the group is four or five members, which are considered ideal sizes. A few remarks about stability:

- It is unknown if it is always possible to have perfectly stable solutions for this kind of problem.
- Stability is not deterministic, there might be different solutions that are equally stable.
- It is possible the students may have a different weight to each of the preferences. But, in this research, it is considered that all of them have the same weight to all their preferences.
- It is also possible that several members of different groups want to get out from their respective groups and form a new group because it suits more their preferences. However, looking for these could lead to an exhaustive search among all the possible configurations just to calculate the stability, beating the purpose of using meta-heuristics.
- This research does not try to reach perfectly stable solutions, but rather near-stable solutions.

3.4 Operators

This next section describes the different operators applied by the algorithms to the solutions. In the case of meta-heuristics, like LS or PT, wherein each iteration a step in the search space is performed, the mutation operator is used. For the case of RD, which requires all a defined neighbourhood, all the possible mutations are used. For the GA and ES, a crossover operator is also used. The same mutation and crossover operators are used for all the algorithms that allow them.

3.4.1 Initialisation

All the algorithms start with a random initial population. First, the groups are generated according to the maximum possible groups, which is ($\lfloor \frac{n}{gs_{min}} \rfloor$). These groups are stored in a

vector for the available groups (AG). When a group is filled, it means it has reached gs_{max} students. Therefore the group leaves vector AG, if a move is performed by the operators and the group misses a student, then this group returns to the AG vector.

Then each student is assigned to a random group from AG. A solution becomes unfeasible if a student is left alone in a group. To prevent unfeasibility, a $Repair$ operator, described in Section 3.4.4, is applied after the initialisation. The initialisation process is described in Algorithm 10.

```
AG ← Create (⌊n / gs_min⌋) groups;
for s_i ∈ D do
    s_i ← G ∈ AG;
    // Assign a random group to s_i
    A ← s_i;
end
Perform Repair in A;
```

Algorithm 10: Initialisation

3.4.2 Mutation

The mutation operator must introduce solutions that are not present in the system yet, in order to avoid getting stuck in a local minimum. A way to define this operator for a combinatorial problem would consist of changing the value of a variable. Since we are considering the students as our variables and the groups as our values, the mutation simply changes the assignation of a group of a particular random user to another group.

To prevent a student ending up in a group that has more students than gs_{max}, a random group is selected from AG which guarantees to produce only feasible solutions. The resulting algorithm is known as group-swap mutation because it swaps the group for a single user once the mutation rate has met. Group swap mutation is described in Algorithm 11. An example of this can be seen in Figure 3.3.

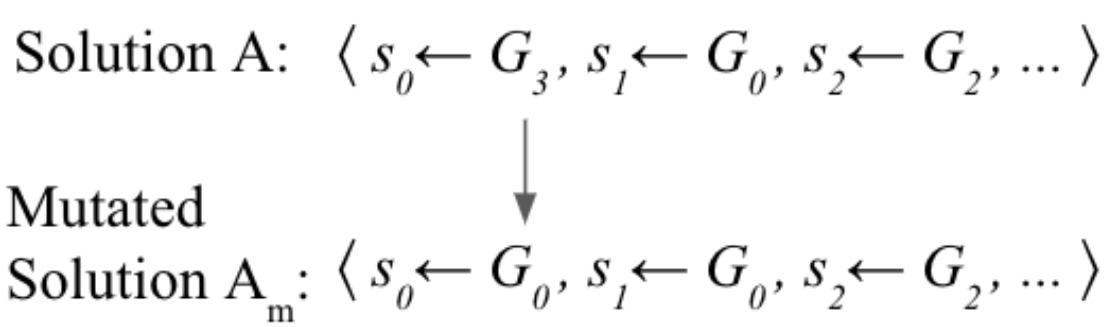


Figure 3.3: An example of a mutation of a solution A.

```
for s_i ∈ A do
    if random() ¡ mutation_rate then
        s_i ← G ∈ AG;
    end
end
```

Algorithm 11: Group Swap Mutation

3.4.3 Crossover

The crossover operator is meant to add convergence to already good solutions, so their offspring can reach a local minimum. The crossover used in this work is the k-point crossover [41], which puts a random number of crossover points to indicate where the solution will be split in two parts and have these parts recombined with another solution. An example of this can be seen in Figure 3.4, where a crossover point is present in the middle. In terms of this research, it means that a portion of the students will remain in the group defined by their parent solution and the rest will come from another parent solution. Hence, good solutions can explore part of their good results with their children, but at the same time improve their quality if they land with a similar part of another group.

Crossover point

Solution A = $\langle s_0 \leftarrow G_3, s_1 \leftarrow G_0, s_2 \leftarrow G_2, s_3 \leftarrow G_1, s_4 \leftarrow G_3 \mid s_5 \leftarrow G_1, s_6 \leftarrow G_1, s_7 \leftarrow G_3, s_8 \leftarrow G_3, s_9 \leftarrow G_2 \rangle$

Solution B = $\langle s_0 \leftarrow G_2, s_1 \leftarrow G_1, s_2 \leftarrow G_2, s_3 \leftarrow G_2, s_4 \leftarrow G_1 \mid s_5 \leftarrow G_0, s_6 \leftarrow G_3, s_7 \leftarrow G_3, s_8 \leftarrow G_0, s_9 \leftarrow G_0 \rangle$

Offspring C = $\langle s_0 \leftarrow G_3, s_1 \leftarrow G_0, s_2 \leftarrow G_2, s_3 \leftarrow G_1, s_4 \leftarrow G_3, s_5 \leftarrow G_0, s_6 \leftarrow G_3, s_7 \leftarrow G_3, s_8 \leftarrow G_0, s_9 \leftarrow G_0 \rangle$

Offspring D = $\langle s_0 \leftarrow G_2, s_1 \leftarrow G_1, s_2 \leftarrow G_2, s_3 \leftarrow G_2, s_4 \leftarrow G_1, s_5 \leftarrow G_1, s_6 \leftarrow G_1, s_7 \leftarrow G_3, s_8 \leftarrow G_3, s_9 \leftarrow G_2 \rangle$

Figure 3.4: Example of a crossover operation.

It should be noted that, since the assignment is not done with only compatible groups, it is possible that some groups become infeasible. That means that crossover can lead to groups with more students than gs_{max} or less than gs_{min}. To prevent this issue, after the crossover operator is applied a repair of the offspring solutions is performed. The whole process can be seen in Algorithm 12.

```
Select two parents A and B from a parent pool;
Create two offspring C and D as follows:;
Randomly choose k crossover points cp_1, ..., cp_k ∈ {1, ..., n − 1};
for i ← 1 to cp_1 do
    c_i ← a_i;
    d_i ← b_i;
end
switch ← 0;
for j ← 2 to k do
    if switch = 0 then
        for i ← cp_{j−1} + 1 to cp_j do
            c_i ← b_i;
            d_i ← a_i;
        end
        switch ← 1;
    else
        for i ← cp_{j−1} + 1 to cp_j do
            c_i ← a_i;
            d_i ← b_i;
        end
        switch ← 0;
    end
end
if switch = 0 then
    for i ← cp_k + 1 to n do
        c_i ← b_i;
        d_i ← a_i;
    end
else
    for i ← cp_k + 1 to n do
        c_i ← a_i;
        d_i ← b_i;
    end
end
Perform Repair in C and D;
```

Algorithm 12: K-point Crossover

3.4.4 Repair

The defined operators can lead to either a group that ends up with a higher number of students than gs_{max} or a smaller number than gs_{min}. For this reason, a repair procedure is performed after the initial population and after each crossover.

The repair algorithm consists of checking each group size, if the size is higher than gs_{max}, then the group is split in half. On the other hand, if the size is lower than gs_{min}, then it

requires more members to be feasible and is marked as a pending group P_g. Therefore, after a group G_i with $Size(G_i) > gs_{max}$ is found a user from G_i is assigned to P_g, and the algorithm continues until all the groups become feasible. The fill task can be seen in Algorithm 13.

```
A_r = ();
// Initialise a vector that will hold the repaired solution
P_g ← NULL;
for G_i ∈ A do
    if Size(G_i) > 0 then
        if P_g is not NULL AND Size(P_g) ≤ gs_max AND Size(P_g) > gs_min then
            A_r ← MergeGroups(P_g, G_i);
            P_g ← NULL;
        end
        if Size(G_i) < gs_min then
            A_r ← SplitGroup(G_i);
            // Split G_i the group in half
        end
        if Size(G_i) > gs_max then
            P_g ← G_i;
        else
            A_r ← G_i;
        end
    end
end
```

Algorithm 13: Group Repair

3.5 Discussion for the following chapter

This chapter showed the different objective functions to optimise and the definition of the operators to be used. In the following chapter is discussed several exploratory experiments that test these functions and operators. In the case of *Interests Similarity Function* and *Participation Style function*, there is a brief experimentation that lead to their definitions.

Chapter 4

Preliminary Experiments

Before the overall comparison of algorithms took place, it was necessary to test the methodology defined in Chapter 3, each component of the solution model was tested individually, to make sure that the experiments in Chapter 5 ran correctly and the check if the optimisation made sense.

For these experiments, there were three different steps taken. First, some of the objective functions were tested one by one to see if they achieve our requirements. Then, there is a description of how synthetic data-set was created. Finally, some initial experiments regarding the comparison between algorithms took place, as a first approach to test the objective functions and operators, as well as the data-set defined.

4.1 Experiments regarding objective functions

Some of the objective functions used for the research had a very straightforward approach, specifically Lvl and Gs defined in sections 3.2.2 and 3.2.1 respectively. In the Lvl function the standard deviation σ represents how much variety of different levels of experience there is within a group. For function Gs, the subtraction represents the distance to the ideal group size.

To define the objective functions for interests similarity Int and the participation style Ps it was necessary to perform additional experiments. Since this can be accomplished in different ways and it was not known which was the most adequate. These experiments are described in the following subsections.

4.1.1 Interests distance function

For these experiments, four different students were randomly generated with their respective interest set I_i, as it was described in Section 3.2.3. The purpose is to find the distance between I_1 and I_2. Ideally, we want to have a metric that is a continuous value between zero and one, so it does not need normalisation. Several similarity metrics found in [44] and [46] were tested discarding the ones that did not evaluate to a value from zero to one. This is because when there are no common interests between the students, in some of the metrics it leads to a division by zero. In the end, two metrics were selected for comparison, the cosine distance (CD) and Pearson correlation coefficient (PC).

The results of this experiment can be seen in Table 4.2. Here several comparisons of each pair of students are observed, using both CD and PC. With the difference between both, for comparison. This was done using the sets of interests specified in Table 4.1, where each interest is given a normalised distance to each other.

Interests	S_1	S_2	S_3	S_4
Business	0.50	0.00	0.00	0.00
Property	1.00	0.00	0.00	0.00
Entertainment	0.33	0.00	0.00	0.50
Games	0.50	0.00	0.00	0.00
Gambling	1.00	0.00	0.00	0.00
Films	0.50	0.00	0.00	1.00
Animated Films	1.00	0.00	0.00	0.00
Family	0.00	0.50	0.00	0.25
Dating	0.00	1.00	0.00	0.00
Marriage	0.00	0.00	0.00	1.00
Fitness	0.00	0.50	0.00	0.00
Running	0.00	1.00	0.00	0.00
Pets	0.00	0.50	0.00	1.00
Hobbies	0.00	0.33	0.33	0.50
Dogs	0.00	1.00	0.00	0.00
Travel	0.00	0.00	0.50	0.00
Beaches	0.00	0.00	1.00	0.00
Arts	0.00	0.00	0.50	0.00
Guitar	0.00	0.00	1.00	0.00
Dance	0.00	0.00	1.00	0.00

Table 4.1: Sample interests by student.

4.1.2 Participation style Function

The participation style function aims to reduce the *quantity of silence* in the conversations of a group to a minimum. Therefore, the objective function should have a relationship between this *quantity of silence*. In the literature, there is no already a metric or a mathematical function that is able to tell how much silence will be produced in a determined conversation. What it has been found is a model created by Benne and Sheats [9] that assigns a *Functional Role* label to the intention each of the utterances has in a conversation. It is assumed that there must be a correlation between the *quantity of silences* in a conversation and the *Functional Role* or a combination of *Functional Roles* in a conversation.

To understand the behaviour in a conversation, it was proposed to make brief data analysis of a group of conversation and check the interaction between the silence quantity and the functional roles. For this analysis, it was found a well-known data-set AMI meeting corpus [38], which has previously been used in other studies [17, 37]. The data-set includes

Metric	Value
Cosine Similarity	0.00
Pearson + 1	0.56
Pearson	-0.43
CD - Pearson	0.43

(a) Comparison between S_1 and S_2.

Metric	Value
Cosine Similarity	0.00
Pearson + 1	0.60
Pearson	-0.39
CD - Pearson	0.39

(b) Comparison between S_1 and S_3.

Metric	Value
Cosine Similarity	0.18
Pearson + 1	0.86
Pearson	-0.13
CD - Pearson	0.31

(c) Comparison between S_1 and S_4.

Metric	Value
Cosine Similarity	0.029
Pearson + 1	0.65
Pearson	-0.34
CD - Pearson	0.37

(d) Comparison between S_2 and S_3.

Metric	Value
Cosine Similarity	0.21
Pearson + 1	0.91
Pearson	-0.08
CD - Pearson	0.30

(e) Comparison between S_2 and S_4.

Metric	Value
Cosine Similarity	0.04
Pearson + 1	0.71
Pearson	-0.28
CD - Pearson	0.32

(f) Comparison between S_3 and S_4.

Table 4.2: Comparison between the sample students.

several conversations of different meetings with different people, with markers of time indicating when each utterance occur, along with with some notes and an accompanying video. Some of the meetings found in this data-set had also been labelled with their correspondent *Functional Role* for each utterance by [17].

The meetings used for these analysis have the following names: ”ES2002d”, ”ES2008b”, ”ES2008d”, ”ES2009d” and ”IS1003d”. The first task was to analyse the *quantity of silences* and their duration. Fortunately, each utterance also has a start time and an end time, so the proportion of the conversation which was silence is considered as the time in which no utterance is occurring.

For each meeting, the total silence time in seconds was considered as the proportion corresponding to the total conversation time. The results can be seen in Table 4.3. Here, it can be observed the number of silences in seconds and the total time each conversation was in silence proportionally. It can be observed, for example, that the meeting IS1003d had less silence time in comparison to ES2008d.

	Total Silence time	Silence proportion
ES2002d	$6861.49s$	72.89%
ES2008b	$7867.38s$	75.24%
ES2008d	$6973.49s$	78.23%
ES2009d	$5963.49s$	70.49%
IS1003d	$5514.33s$	66.78%

Table 4.3: Quantity of silences and their proportion by meeting.

One of the conversation functional roles of Benne and Sheats is described as ”protagonism”. This infers that a person making an utterance with this intention is starting to talk about a specific topic, not just responding by the utterance made by others. It was inferred that this specific role may have some correlation with the lack of silences. The next step involved measuring the *protagonistic proportion* and *protagonistic time* of each of the participants in the conversation. This is for all the utterance they made, how many were *protagonistic* and for all the time they spoke, how much of it was *protagonistic* respectively.

Tables 4.5 and 4.4 show the *protagonistic proportion* and *protagonistic time* for each participant in the conversation with the most (ES2008d) and the less (IS1003d) *quantity of silences* respectively. As it can be seen in Table 4.5, the meeting ES2008d has only brief protagonisms by a single person in the conversation, while in Table 4.4, the meeting IS1003d has a homogeneous amount of *protagonisms* by all of their participants.

Participant	Protagonism proportion	Protagonistic time
A	9.46%	3.97%
B	0.58%	0.23%
C	4.19%	1.86%
D	15.9%	5.16%

Table 4.4: Protagonism proportion for the Meeting IS1003d.

Participant	Protagonism proportion	Protagonistic time
A	9.29%	3.2%
B	0.0%	0.0%
C	0.0%	0.0%
D	0.0%	0.0%

Table 4.5: Protagonism proportion for the Meeting ES2008d.

Finally, the number of participations was compared to the average participation and the summation of these participations, shown in Table 4.6. Here we proposed an inverse correlation to the sum of participations $\frac{1}{\sigma}$ as the metric for improving the number of participations and therefore lowering the *quantity of silence*, as it can be seen in Table 4.6, a low value of it represents a positive impact on the number of participations.

Conversation	Participations proportion	Average of paticipation	Σ	$\frac{1}{\Sigma}$
ES2008b	24.75%	0.80%	23.17%	04.30%
ES2008d	24.16%	5.79%	03.21%	31.17%
ES2002d	27.11%	4.29%	17.15%	05.83%
ES2009d	29.51%	2.43%	09.72%	10.29%
IS1003d	33.21%	2.81%	11.23%	08.90%

Table 4.6: Results of the conversation analysis.

It should be noted that this, by no means, is considered a strong data analysis, nor the objective function should be considered as definitive. But, as a first approach, and considering the information that could be gathered at the moment, it serves its purpose for the sake of this research.

4.2 Synthetic data-set creation

To calculate the value for each objective function described in Section 3.2, it is necessary to consider some features about each student regarding their experience level, their interests and their participation style.

The experience level is represented as a numerical quantity. For the interests, it was considered a fixed size of three interests for each one of the students, and a feature representing the participation style was considered as the percentage of the Benne and Sheats role [9] of ”Protagonist” that a student has on average in a conversation.

Since these features have never been used in any other scenario together, there was not a specific data-set that included all of them. Therefore, the data-set was created in such a way that it resembles what a real data-set would look like once data is gathered for each of the features. To this end, the mean and standard deviation were used to determine the statistical distributions. In the following subsections is discussed how data was obtained and other important details for the data-set creation.

In the following subsection is described where the metrics (mean and standard deviation) comes from and why were they used.

4.2.1 Experience Level

While there is no distribution information for the language levels for CEFR specifically, the IELTS has data about the frequency distributions by percentage [2] which has a distribution by a given level. Fortunately, these levels have their equivalence in the CEFR scale, as can be observed in Table 4.7.

CEFR Level	Proportion
A0	0.33%
A1	0.33%
A2	0.33%
B1	13.0%
B2	48.0%
C1	36.0%
C2	2.0%

Table 4.7: The distribution by each level.

4.2.2 Interests

The social network Facebook has a database of interests separated by population and regions. This data corresponds to the ontology presented in Section 2.8.2 and Section 3.2.3. The main purpose of the Facebook Ad platform [1] is to build marketing campaigns.

When a user of this platform consults the potential marketing segment, filtered by specific interest, a number known as *potential reach* is shown. This number was acquired for each of the interests in the ontology to figure out the proportion of the population for each interest.

The ontology contains overall categories (such as ”Business and Industry” or ”Hobbies”). This, however, was not considered for the distribution estimation, because they represented very general interests. Therefore, only interests from the third level onwards in the ontology were considered. Part of the distributions of interests can be seen in Table 4.8.

Interest name	Proportion
Television programme	62.94%
Restaurants	45.88%
Vehicles	27.25%
Sports	13.00%
Music	8.21%
Arts and music	5.29%
Travel	5.11%
Films	4.61%
Reading	4.51%
Consumer electronics	4.43%
Games	4.18%
Shopping	3.68%
Beauty	3.49%
Live events	3.21%
Politics and social issues	2.96%
Food	2.87%
Computers	2.48%
Pets	2.29%
Clothing	2.07%
Drinks	2.04%

Table 4.8: The first 20 Interests and their proportion.

4.2.3 Participation style

For the participation style, the same data-set as for defining the function was used, the AMI meeting corpus. Since we established the metric to be used as the *proportion of protagonisms*, the mean and standard deviation were calculated directly per person in each conversation. The results of this can be seen in Table 4.9. Here, the mean and standard deviation of the *proportion of protagonisms* for each of the participants in each of the conversations is shown.

Protagonism mean	9.67%
Protagonism standard deviation	6.66%

Table 4.9: Results for the Participation Analysis.

4.3 Exploratory Experiments

Before comparing the proposed algorithms, several exploratory experiments took place to understand how the objective functions and the proposed operators behave. This also serves as the first comparison of a simple meta-heuristic, a population-based meta-heuristic, and a multi-objective algorithm. All algorithms were subject to similar parameters, described as follows.

- A data-set of $n = 10,001$ was used with the mutation operator described in Section 3.4.2 at a $1/n$ rate with n representing the number of students.
- For the single objective algorithms, the same weight for all the functions was considered.
- For the population-based algorithms GA and NSGA-II the crossover operator defined in Section 3.4.3 was used at a 0.9 rate.
- Also, for the population-based algorithms, a population of $p = 40$ was considered.
- All the algorithms ran for over $10,000$ evaluations, which means $10,000$ steps for LS, and 250 generations for GA and NSGA-II

4.3.1 Single-Objective Optimisation

For the SOA, a Weighted Sum was used, shown in Equation 4.1, considering each of the objective functions previously defined. This also serves as the basis for comparison between these single objective and multi-objective experiments.

- Function for the group size f_1
- Function for the interests similarity f_2
- Function for the similarity of experience level f_3
- Function for the participation style f_4

$$f_{agg} = \sum_{i=1}^{4} \omega_i f_i \tag{4.1}$$

Local Search

For the first experiment, a LS algorithm was used, due to is one of the most simple metaheuristics for single-objective optimisation according to the literature [10]. As it can be seen in Figure 4.1, from a random solution the algorithm manages to improve until it gets stuck in a local minimum.

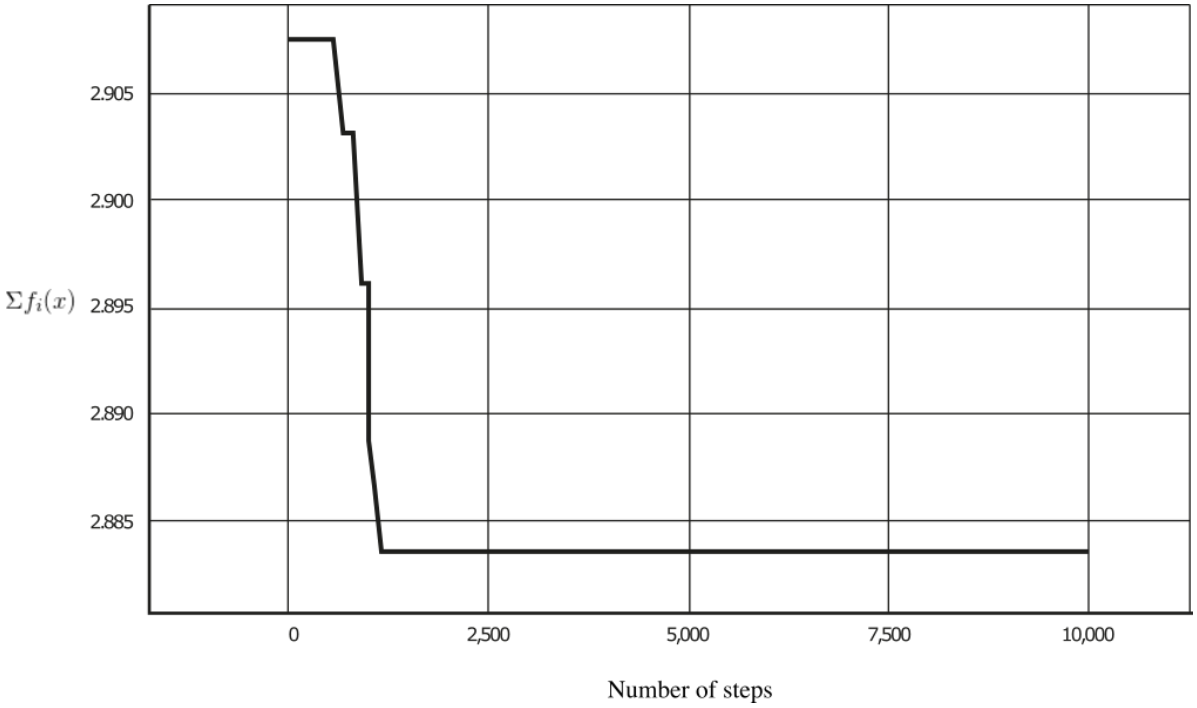


Figure 4.1: Performance of the LS algorithm.

Genetic Algorithm

LS is a meta-heuristic that only makes a single change per step. However, it was also necessary to test the proposed crossover operator in a population-based approach. An experiment using a GA was also tested with similar parameters to LS, with the number of iterations corresponding to the number of generations. As can be seen in Figure 4.2, the algorithm manages to improve for the first 100 steps until it reaches a local minimum, then it continues moving to worse solutions.

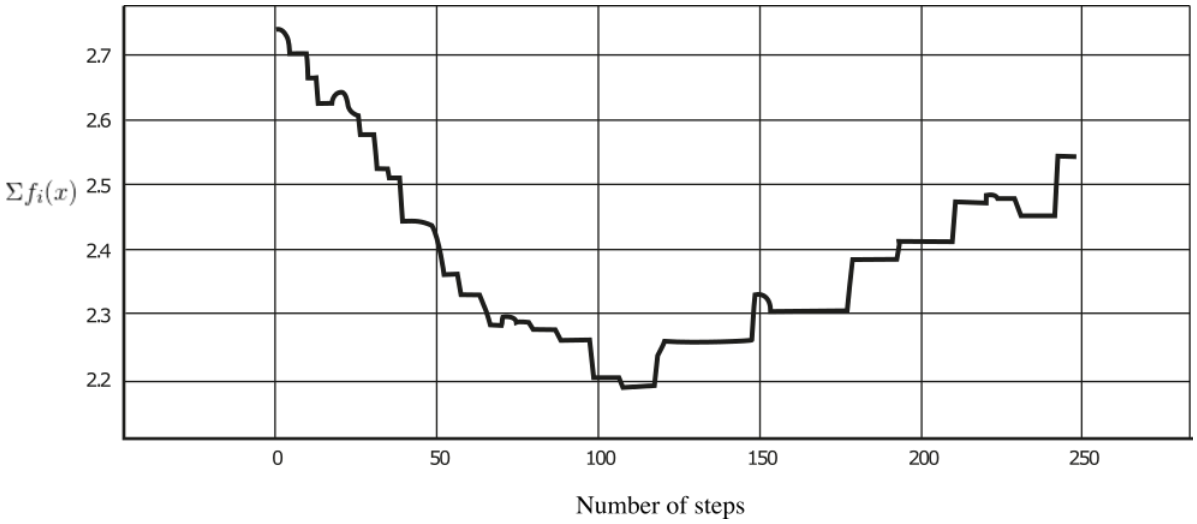


Figure 4.2: Performance of the single objective GA.

4.3.2 Multi-Objective Algorithm

For the single-objective optimisation, it was decided to use NSGA-II, since is one of the most common algorithms found in the literature and has mostly the same parameters as the single objective genetic algorithm.

To compare it to the single-objective algorithms, f_{agg} was evaluated for each of the solutions in the obtained front in each step. The graph in Figure 4.3 is made from the best solutions from the fronts produced in each step considering f_{agg}.

As can be seen in Figure 4.3 the algorithm is stuck at several local minima, but it manages to continue improving. It also should be noticed that it improves significantly better than its single-objective counterparts.

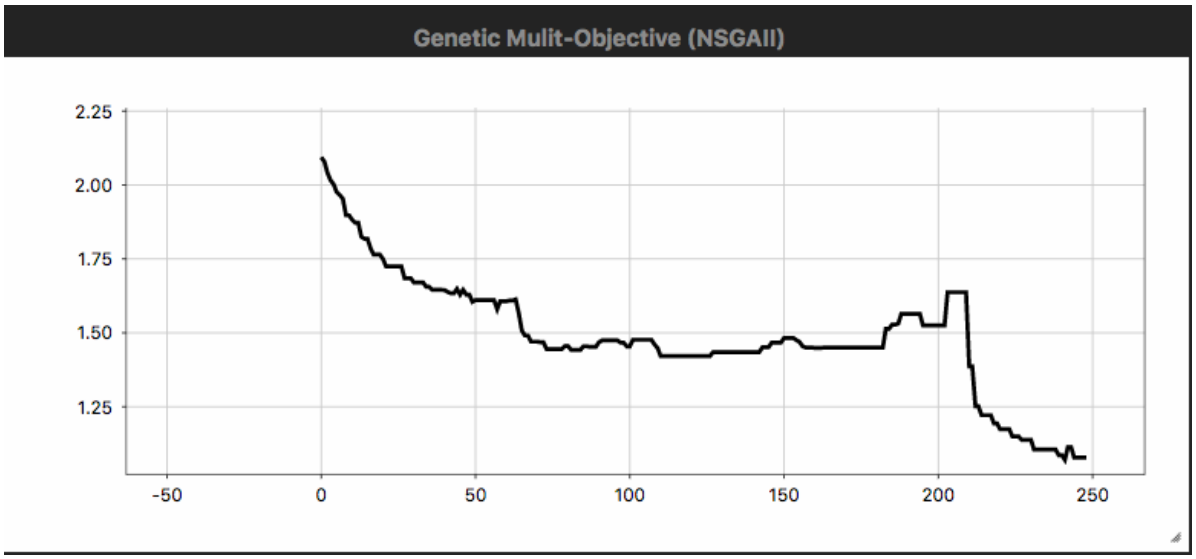


Figure 4.3: Performance of the NSGA-II.

4.4 Preliminary conclusions

After analysing the results, it can be noticed that there is very little variation with the aggregated function f_{agg}, this is mostly because for this experimentation phase the same weights were considered for all the functions. However, considering that not all functions have the same minimum and maximum values, there was a clear need to normalise them. This is done in the experiments in Chapter 5. It should also be noticed that for the LS algorithm f_{agg} seems to become trapped in the local minima, further evidence of this is GA and NSGA-II being able to find a better solution. Finally, NSGA-II had the best performance overall, which may be a clue that MOA may be able to outperform SOA. It should also be noticed that despite using a large data-set of 10,001 students for these experiments, the local minima appear after 100 steps.

Chapter 5

Experiments and Results

This chapter presents the core experiments of the research. The main purpose of it was to do a benchmark, as a starting point for the comparison between SOAs and MOAs. The basis for the experimentation layout was based on the experimental framework proposed by H. Ishibuchi, Y. Nojima and T. Doi [30].

The chapter is divided into four different sections, described below:

- **Experiments setup** consists of a set of experiments to determine a reference front to use it for comparison against SOAs and MOAs.
- **First phase** consists of the run of the algorithms without any adjustment using the parameters found in the experimental setup for all the algorithms. This also includes an analysis of the stability of some of the resulting groups.
- **Second phase** takes into account the results of the first phase of experiments and feedback made for its results. It discusses one of the main issues made by the feedback which remarks on whether or not the comparison between SOA and MOA was fair.
- **Third phase** is the result of a discussion about the tuning of parameters. Using only the best algorithms from phase one, chosen by their performance using the parameters from the Experimental setup.

5.1 Experiments Setup

The next section discusses a set of experiments that took place before the core experiments could be made. The first subsection shows the experiment made to search for a reference front for the performance comparison. The next subsection shows an experiment for obtaining the base parameters that would be used to have a fair comparison between the algorithms. The last subsection discusses some of the specific details about each of the individual algorithm implementations.

5.1.1 Searching for a Reference Pareto Front

The IGD+ metric requires to have a defined reference front to perform a proper comparison. To this end, the NSGA-II algorithm was used with a population of 20 individuals for 1000 generations, across 200 independent runs. The best front was chosen considering their HV for each of the problem sizes.

After the experiment, the results of each run were compared with each other, and the ones with the lowest HV were chosen. These results are shown in Figures 5.1 to 5.3 for each respective problem. An important observation is that, in n=20 and n=200, some of the individuals had the same value for each of their objective functions. This observation is important because it is an indication that a global minimum for these problems may exist and could be found using an SOA.

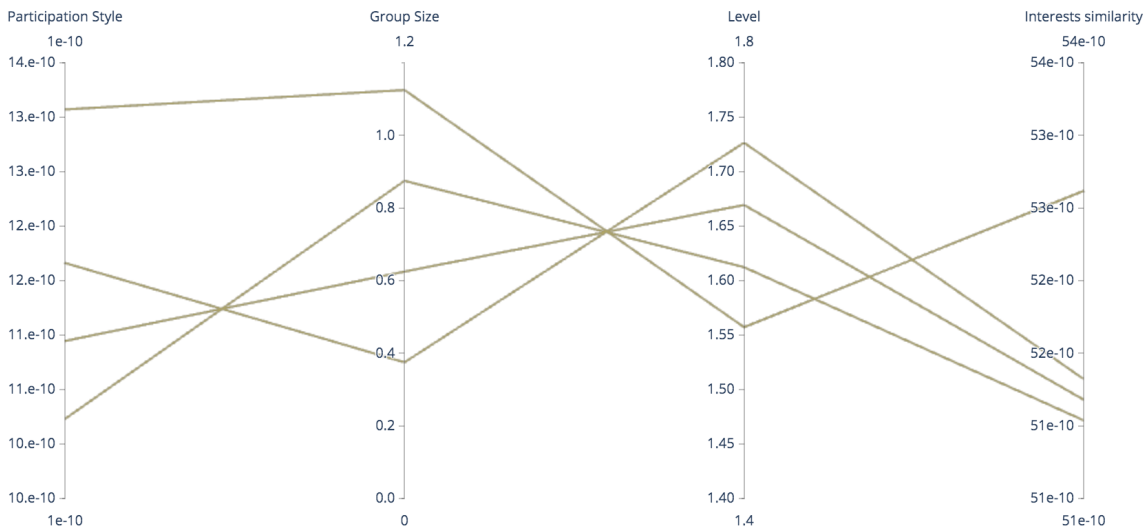


Figure 5.1: Reference front for the $n = 20$ problem.

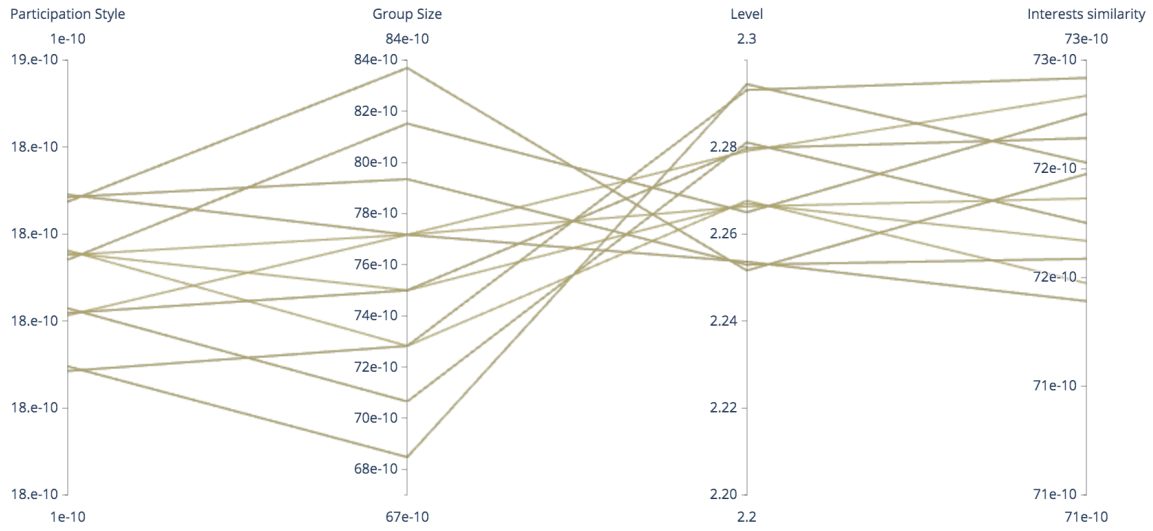


Figure 5.2: Reference front for the $n = 200$ problem.

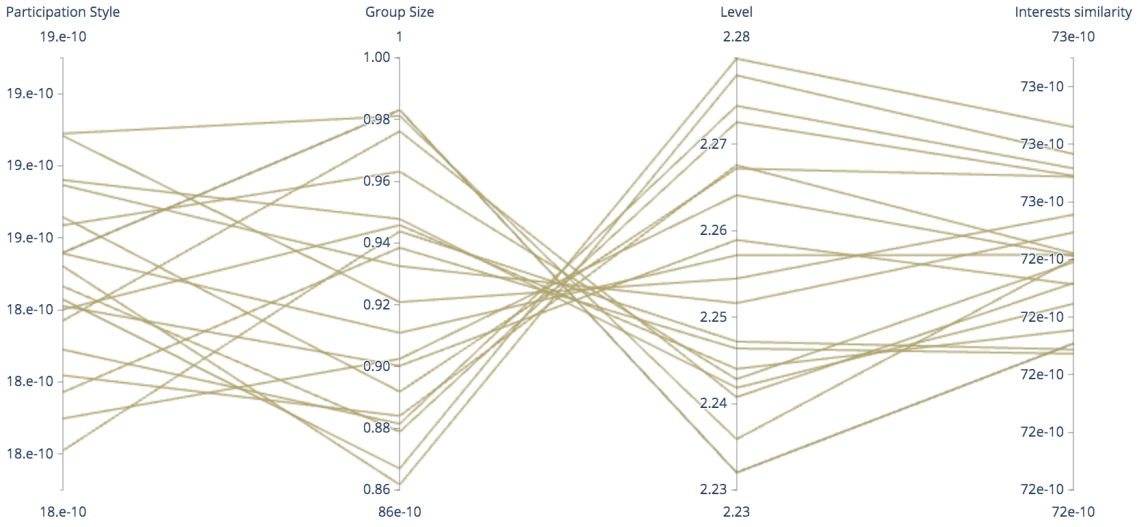


Figure 5.3: Reference front for the $n = 2,000$ problem.

5.1.2 Searching for Reference Parameters

After some empiric testing with several values for the number of generations or steps s and the population size p, there was a significant rise in complexity, especially for the high problem of $n = 2,000$. This resulted in some experiments running for several hours and an overuse of memory space, even after using memorisation for the evaluation of costly objective functions. So, there is a bias to chose a pair small enough to maintain the complexity low, but high enough to produce a significant comparison.

Considering that a fair comparison requires a level of complexity similar to each algorithm, this research used the same parameters for s and p, and the same operators for each of the algorithms. It should be noted that not all the algorithms used both of these parameters, but they were considered because they were shared across most of the algorithms.

In this experiment, several combinations of these parameters were tested for SOA and MOA. For the SOA, a generational GA was used, whereas, for the MOA, the algorithm NSGA-II was used. These pairs of parameters (p, s) to be tested were established on a constant scale: **100**, **150**, **200**, **250**, and **300**. This experiment only considered the problem sizes of **20** and **200** students. Each pair of parameters was tested for **50** independent runs. Although a high performance may be observed from the high parameters, it should be considered that a pair with lower complexity is preferred.

	Best (p, s)	Best Value	Second Best (p, s)	Second Best Value
EP Mean and STD	150,250	$8.77_{3.50}$	100,150	$8.98_{3.90}$
EP Med and Inter Range	250,250	$9.45_{2.90}$	300,150	$9.46_{3.40}$
SPREAD Mean and STD	100,100	$1.00_{0.00}$	150,100	$1.00_{0.00}$
SPREAD Med and Inter Range	100,100	$1.00_{0.00}$	150,100	$1.00_{0.00}$
GD Mean and STD	150,250	$9.27_{3.90}$	250,250	$9.29_{3.60}$
GD Med and Inter Range	250,300	$9.84_{3.70}$	250,250	$9.92_{3.60}$
IGD Mean and STD	150,250	$2.25_{0.90}$	250,250	$2.25_{0.83}$
IGD Med and Inter Range	250,300	$2.40_{0.83}$	250,250	$2.42_{0.88}$
IGD+ Mean and STD	150,250	$9.80_{4.30}$	250,250	$9.93_{3.9}$
IGD+ Med and Inter Range	250,300	$10.07_{3.70}$	250,250	$10.08_{3.9}$

Table 5.1: The results of the parameters experiment of $n = 20$ for SOA, by combinations of a population size p and a number of steps or generations s.

	Best (p, s)	Best Value	Second Best (p, s)	Second Best Value
EP Mean and STD	100,100	$30.97_{5.60}$	200,150	$30.99_{5.40}$
EP Med and Inter Range	200,150	$30.94_{7.60}$	250,300	$30.96_{8.00}$
SPREAD Mean and STD	100,100	$1.00_{0.00}$	150,100	$1.00_{0.00}$
SPREAD Med and Inter Range	100,100	$1.00_{0.00}$	150,100	$1.00_{0.00}$
GD Mean and STD	100,100	$4.04_{5.80}$	200,150	$40.05_{5.60}$
GD Med and Inter Range	200,150	$40.02_{7.80}$	250,300	$40.03_{7.90}$
IGD Mean and STD	100,100	$9.16_{1.30}$	200,150	$9.18_{1.20}$
IGD Med and Inter Range	200,150	$9.12_{1.70}$	250,300	$9.15_{1.70}$
IGD+ Mean and STD	100,100	$40.00_{5.90}$	200,150	$40.01_{5.70}$
IGD+ Med and Inter Range	200,150	$30.96_{7.90}$	250,300	$30.98_{8.40}$

Table 5.2: The results of the parameters experiment of $n = 200$ for SOA, by combinations of a population size p and a number of steps or generations s.

	Best (p, s)	Best Value	Second Best (p, s)	Second Best Value
EP Mean and STD	100,300	$2.83_{2.2}$	300,100	$3.08_{2.4}$
EP Med and Inter Range	200,300	$1.63_{4.3}$	100,300	$1.69_{4.0}$
SPREAD Mean and STD	100,100	$1.00_{0.0}$	150,100	$1.00_{0.0}$
SPREAD Med and Inter Range	100,100	$1.00_{0.0}$	150,101	$1.00_{0.0}$
GD Mean and STD	100,300	$0.14_{0.14}$	300,300	$0.18_{0.18}$
GD Med and Inter Range	100,300	$0.08_{0.24}$	200,300	$0.08_{0.29}$
IGD Mean and STD	100,300	$0.71_{0.54}$	300,100	$0.75_{0.59}$
IGD Med and Inter Range	100,300	$0.41_{0.89}$	300,100	$0.41_{0.98}$
IGD+ Mean and STD	100,300	$2.89_{2.5}$	300,100	$3.19_{2.7}$
IGD+ Med and Inter Range	200,300	$1.48_{4.5}$	100,250	$1.64_{7.1}$
HV Mean and STD	150,150	$0.36_{64.00}$	100,100	$35.1_{59.00}$
HV Med and Inter Range	200,100	$9.78_{40.50}$	300,200	$6.20_{34.00}$

Table 5.3: The results of the parameters experiment of $n = 20$ for MOA, by combinations of a population size p and a number of steps or generations s.

	Best (p, s)	Best Value	Second Best (p, s)	Second Best Value
EP Mean and STD	100,100	$5.42_{2.30}$	100,200	$6.37_{3.30}$
EP Med and Inter Range	100,100	$5.32_{2.80}$	100,200	$5.64_{5.30}$
SPREAD Mean and STD	250,300	$1.00_{0.00}$	300,300	$1.00_{0.00}$
SPREAD Med and Inter Range	150,100	$1.00_{0.00}$	200,100	$1.00_{0.00}$
GD Mean and STD	100,200	$0.46_{0.24}$	100,300	$0.50_{0.23}$
GD Med and Inter Range	100,200	$0.40_{0.35}$	100,150	$0.50_{0.51}$
IGD Mean and STD	100,100	$1.24_{0.50}$	100,150	$1.50_{0.74}$
IGD Med and Inter Range	100,100	$1.18_{0.62}$	100,200	$1.34_{1.20}$
IGD+ Mean and STD	100,100	$5.33_{2.30}$	100,150	$6.37_{3.50}$
IGD+ Med and Inter Range	100,100	$5.21_{3.00}$	100,200	$5.47_{5.20}$
HV Mean and STD	300,200	$440.00_{570.00}$	300,250	$449.00_{640.00}$
HV Med and Inter Range	300,150	$207.00_{420.00}$	200,250	$239.00_{390.00}$

Table 5.4: The results of the parameters experiment of $n = 200$ for MOA, by combinations of a population size p and a number of steps or generations s.

5.2 Implementation Details

In this section, the descriptions for all the algorithms are given with each of their specific implementations for the problem. As previously noted, all the implementations were already included in the **jMetal** and **JAMES** frameworks, and the only extra functions created were the mutation and crossover operators which were detailed in Section **??**.

5.2.1 Single-Objective Algorithms

A disadvantage found in the experiments of Chapter 4 was that for the SOA, the ranges of each of the objective functions varied considerably, which added more weight for the higher evaluation results. This issue is addressed using *a posteriori* normalisation of the parameters,

which are computed with Equation (5.1) based on the bounds given in Table 5.5.

The full details of each of the implementations can be seen in Table 5.6 for the algorithms used present in the jMetal framework. Table 5.7 presents for the algorithms used in the James framework.

$$v_{norm} = \frac{v - v_{min}}{v_{max} - v_{vmin}} \tag{5.1}$$

Objective function	Minimum value v_{min}	Maximum value v_{max}
Group Size Function	0.50	1.50
Participation Style Function	0.00	1.00
Level Function	0.00	2.82
Interests Cosine Similarity Function	0.00	1.00

Table 5.5: The v_{min} and v_{max} values for each objective function used in the normalisation.

Denomination	Full name	Details	Additional Parameters
GGA	Genetic Generational Algorithm	Is an implementation of the Genetic Algorithm in which all the solutions are replaced in each of the generations.	n/a
GSA	Genetic Steady Algorithm	Is an implementation of the Genetic Algorithm in which a single solution is added and another one is eliminated in each generation	n/a
ES	Elitist Strategy Algorithm $(\mu + \lambda)$	Randomly selects a parent form the set of μ individuals from both the parents and offspring of the last generation, the parent is then mutated and generates λ offsprings.	$\mu = 1$; $\lambda = pop(100)$
NES	Non-Elitist Strategy Algorithm (μ, λ)	Randomly selects a parent form the set of μ individuals from both the parents and offspring of the last generation, and the parent is then mutated and generates λ offsprings	$\lambda = pop(100)$
LS	Local Search (Steepest Descent/Hill Climbing)	The implementation found in the framework jMetal, was the one used as a middle step procedure for other algorithms like $ABYSS$. it makes use of a Dominance comparator	$\epsilon = 0$
SORS	Random Search	n/a	n/a

Table 5.6: Specific details of the Single-Objective Algorithms Implementations based on the jMetal Framework.

Denomination	Full name	Details	Additional Parameters
RD	Random Descent	The neighbourhood is considered as a mutation step, so all the possible mutations are considered in each step.	n/a
PT	Parallel Tempering (Replica Exchange Monte Carlo)	Similar to Simulated Annealing this algorithm uses a max temperature and a low temperature, also running parallel replicas of Metropolis search.	$temp_{min} = 1 * 1e^{-8}$;$temp_{max} = 1 * 0.6$; $steps_{max} = 100$; $replicas = 2$

Table 5.7: Specific details of the Single-Objective Algorithms Implementations based on the James Framework.

Multi-Objective algorithms

The details and specific parameters for the multi-objective algorithms can be seen in Table 5.8. In this case, all the algorithms implementations belong to the jMetal framework. It should also be noted that a binary tournament selection, a ranking, and crowding distance were used for all these algorithms.

Denomination	Full Name	Details	Additional parameters
ESPEA	Electrostatic Potential Energy Evolutionary Algorithm	Since no scalarized preference is specified, uniform preferences are assumed Uses a Worst in Archive Replacement Strategy. Where Among all eligible archive members that can be replaced the one exhibiting the largest energy contribution is replaced.	Replacement Strategy: Worst in Archive
MOMBI-II	Many Objective Metaheuristic Based on the R2 Indicator	Requires a number of weights equal to the population size. These weights were selected according to the population size using the Simplex-Lattice Design method	Vector of weights, using Simple-Lattice Design method; According to the population size.
NSGA-II	Non-dominated Sorting Genetic Algorithm	n/a	n/a
MORS	Multi-Objective Random Search	n/a	n/a
SPEA-II	Strength Pareto Evolutionary Algorithm	n/a	n/a

Table 5.8: Specific details of the Multi-Objective Algorithms Implementations.

5.3 Scalarization Methods

Multi-objective optimisation problems are classically solved using scalarization techniques [21]. Scalarization means that the objective functions will be aggregated or reformed as constraints, therefore a single-objecttive problem is solved. For this research, we make use of

three different scalarization methods: weighted sum, Chebyshev scalarization and ϵ-constraint method which are described in this section.

5.3.1 Weighted Sum Method

Weighted sum (WS) also known as linear weighting is one of the most common ways to scalarize a multi-objective problem [14]. Each objective function is

5.3.2 Chebyshev Scalarization

5.3.3 ϵ-constraints Scalarization

5.4 Experimental Phases

In this section, we explore the performance of the different proposed algorithms for minimising the defined objective functions described in Section 3.2. Specially the differences of performance among SOA and MOA. Each SOA was run using different scalarization methods: WS, CHE and E-Constraints described in Section 5.3.

The experiments are divided in two phases. The main difference between these experiments is the parameters for the mutation rate and the crossover rate. The first phase make use of Typical parameters, these are parameters usually found in the literature, which states a crossover probability between 0.5 and 1.0 and a mutation probability between 0.001 and 0.05 [35]. Specifically for our experiments we use a crossover probability of 0.9 and a mutation probability of $1/n$ where n is the size of the problem which corresponds to the number of students. The second phase implements a more aggressive mutation based on the parameters for solving SMP found in the literature [7]. The problem sizes considered were 20, 200 and $2,000$.

The Stopping condition for all of the algorithms is determined by the number of maximum evaluations which is determined as $Evaluations_{max} = p * s$, where p is the size of the population and s is the number of steps, both defined in Section 5.1.2.

The SOA used were the implementations found in jMetal 5.4 for the GGA, GSA, ES, NES, LS and RS algorithms and RD and PT found in the JAMES 3.4.0 framework, as established in Tables 5.6, 5.7. The MOA used were the implementations of ESPEA, MOMBI-II, NSGA-II, MORS and SPEA-II found in the same jMetal framework, defined in Table 5.8. The executions were done over a GNU/Linux machine with an Intel i7-4790 3.60GHz processor and 16Gb of RAM.

To evaluate the performance we made an analysis using the EPSILON, SPREAD, IGD+ and HV metrics. For the analysis we made use of a Median and Interquartile analysis in order to view which problem sizes may benefit a certain algorithm and a Friedman test is also conducted to determine the best algorithm overall according to each metric. Several graphs are also shown to check the consistency of the results. We performed 30 independent runs for all

the problem sizes.

5.4.1 Scalarization Weights

The EPS scalarization method make use of a boundary to determine its constraints. This boundary was established using all the SOA and optimising each objective function independently for each of the problem sizes, taking the best result for each objective function and for each problem size. The results are shown in Table 5.9. These were used as the boundaries for the constraints, in each step a single objective function is evaluated and the rest are used as constraints. If a function evaluation is higher than the best values it is consider that it had violated the constraint to a certain degree depending on how much higher it was.

n	Gs	Ps	Lvl	Int
20	0.50	0.02	0.67	0.55
200	0.68	0.02	0.67	0.63
2,000	0.86	0.02	0.69	0.65

Table 5.9: The boundaries for each problem size and each function based on the best found results

For the WS and CHE methods there was a need to define a weight for each function, and ideally the sum of them should equal to one. The weights to be used can also make easier to find the Pareto front by dragging the solutions towards its shape [21]. For this reason, we make use of the previous best found values in Table 5.9 normalised, so they add up to one. The normalisation used was the following:

$$w_i = \frac{f_i(x)}{\sum f(x)} \tag{5.2}$$

Where w_i is the weight to be used for each function, $f_i(x)$ is the best result found for this function evaluated individually and $\sum f(x)$ is the sum of all the results for each problem size. The weights are shown in Table 5.10.

n	Gs	Ps	Lvl	Int
20	0.285	0.012	0.385	0.317
200	0.339	0.011	0.333	0.316
2,000	0.386	0.011	0.311	0.292

Table 5.10

Interestingly, these weights are close from each other from their problem sizes. This could mean that there can be global weights to be used for any problem size. But for this

research each problem size uses a different set of weights.

5.4.2 First phase: Thumb Parameters

The common parameters for the mutation and crossover probabilities specify to have a low mutation probability usually between 0.001 and 0.05 and a high crossover probability between 0.5 and 1.0 [35]. In this research, we call these parameters ”Thumb-Parameters”, as they are rules of thumb to follow. For this experimentation phase, we used a crossover probability of 0.9 and a mutation probability of $1/n$ where n is the size of the problem which corresponds to the number of students.

	n = 20			n = 200			n = 2,0000		
	WS	CHE	EPS	WS	CHE	EPS	WS	CHE	EPS
GGA	$22.77_{9.82}$	$5.18_{2.42}$	$6.21_{3.19}$	$75.90_{83.62}$	$14.45_{3.95}$	$28.27_{10.41}$	$27.08_{12.56}$	$7.32_{0.59}$	$8.80_{4.31}$
GSA	$25.46_{13.54}$	$5.98_{1.74}$	$8.49_{2.10}$	$110.45_{136.11}$	$15.31_{4.12}$	$26.00_{9.67}$	$43.36_{19.81}$	$7.24_{0.39}$	$10.04_{1.77}$
ES	$22.62_{14.96}$	$5.12_{3.38}$	$8.95_{2.73}$	$55.90_{27.27}$	$16.14_{3.08}$	$20.64_{6.92}$	$7.97_{0.89}$	$7.56_{0.62}$	$7.62_{0.74}$
NES	$25.49_{10.84}$	$5.29_{3.47}$	$8.36_{2.38}$	$61.79_{63.46}$	$15.74_{2.79}$	$20.04_{4.73}$	$8.24_{1.46}$	$7.54_{0.73}$	$7.59_{0.77}$
LS	$12.94_{7.16}$	$6.73_{1.95}$	$6.40_{1.72}$	$15.66_{5.67}$	$17.08_{4.59}$	$16.81_{4.51}$	$7.60_{0.55}$	$7.51_{0.67}$	$7.51_{0.77}$
RD	$10.66_{5.50}$	$6.32_{1.92}$	$5.73_{1.95}$	$18.19_{2.59}$	$16.97_{5.39}$	$15.84_{5.05}$	$7.40_{0.49}$	$7.51_{0.65}$	$7.58_{0.63}$
PT	$22.48_{10.26}$	$4.33_{2.40}$	$4.73_{2.13}$	$30.65_{15.33}$	$15.52_{3.80}$	$21.24_{8.34}$	$7.76_{0.58}$	$7.51_{0.64}$	$7.50_{0.73}$
SORS	$12.60_{5.61}$	$6.43_{2.93}$	$5.81_{1.82}$	$17.43_{4.24}$	$13.39_{1.77}$	$18.22_{6.02}$	$7.84_{0.93}$	$7.19_{0.71}$	$7.69_{0.57}$
ESPEA		$3.17_{1.10}$			$13.51_{3.75}$			$6.28_{0.36}$	
MOMBI-II		$2.81_{1.46}$			$6.03_{6.81}$			$6.91_{0.40}$	
NSGA-II		$3.02_{1.48}$			$13.13_{1.79}$			$6.41_{0.27}$	
SPEA-II		$2.82_{1.43}$			$11.93_{4.19}$			$6.32_{0.42}$	
MORS		$2.25_{1.28}$			$6.89_{1.15}$			$5.67_{0.05}$	

Table 5.11: Median and Interquartile Range for the IGD+. Metric using Thumb-Parameters

Tables 5.13 and 5.14 show the results for the SPREAD and HV metrics respectively. These are only evaluated for the MOA since the SOA only has a single point in the front per run, therefore for them the spread would always be one. The HV is calculated using the inverse of the front, so a higher HV is considered better.

The two best algorithms according to the HV metric are NSGA-II and SPEA-II. These results are mostly consist ant between problem sizes. These results are mostly consistent with the SPREAD showed in Table 5.13. EP and IGD+ highlighted MORS as the better algorithm, but as previously noted, the difference between the others in comparison wasn’t much. Therefore the analysis of HV showed clearer results among the MOA. While EP and IGD+ showed an slight advantage for the MOA against the SOA in general.

A Friedman ranking test was done to define which algorithms performed better across all problem sizes. The results for the ranking test on the IGD+ and EP metrics are shown in Tables 5.15 and 5.16. The ranking for both of them are pretty close to each other for SOA and for the MOA is exactly the same. There is a clear advantage for using the CHE method with

	n = 20			n = 200			n = 2,0000		
	WS	CHE	Epsilon	WS	CHE	Epsilon	WS	CHE	Epsilon
GGA	$20.15_{10.89}$	$4.66_{1.81}$	$6.38_{3.11}$	$75.72_{84.29}$	$14.96_{3.95}$	$28.50_{10.41}$	$26.48_{12.93}$	$7.85_{0.59}$	$9.32_{3.81}$
GSA	$23.10_{13.79}$	$5.10_{1.27}$	$8.80_{2.04}$	$109.84_{136.94}$	$15.84_{4.11}$	$26.22_{9.80}$	$42.41_{20.35}$	$7.77_{0.39}$	$10.42_{1.68}$
ES	$21.57_{15.65}$	$4.72_{2.00}$	$9.25_{2.73}$	$56.00_{27.38}$	$16.66_{3.07}$	$21.10_{6.94}$	$8.49_{0.87}$	$8.08_{0.60}$	$8.14_{0.74}$
NES	$24.45_{9.80}$	$4.48_{2.20}$	$8.49_{2.56}$	$62.02_{63.56}$	$16.28_{2.79}$	$20.45_{4.63}$	$8.75_{1.43}$	$8.07_{0.74}$	$8.11_{0.75}$
LS	$12.77_{8.08}$	$6.03_{2.58}$	$5.76_{1.33}$	$16.18_{5.69}$	$17.59_{4.58}$	$17.30_{4.61}$	$8.11_{0.54}$	$8.02_{0.67}$	$8.04_{0.76}$
RD	$10.12_{6.84}$	$5.88_{1.74}$	$5.14_{1.77}$	$18.65_{2.64}$	$17.46_{5.39}$	$16.25_{5.13}$	$7.91_{0.50}$	$8.03_{0.66}$	$8.10_{0.62}$
PT	$22.33_{10.77}$	$4.00_{2.06}$	$5.07_{2.42}$	$30.82_{15.47}$	$16.05_{3.80}$	$21.53_{8.04}$	$8.26_{0.58}$	$8.04_{0.63}$	$8.02_{0.74}$
SORS	$12.08_{6.81}$	$5.64_{2.43}$	$5.18_{1.27}$	$17.87_{4.27}$	$13.93_{1.77}$	$18.72_{5.99}$	$8.31_{0.95}$	$7.72_{0.71}$	$8.20_{0.57}$
ESPEA		$3.49_{1.23}$			$14.00_{3.77}$			$6.81_{0.37}$	
MOMBI-II		$3.09_{1.15}$			$6.57_{7.02}$			$7.44_{0.40}$	
NSGA-II		$3.12_{1.25}$			$13.55_{1.70}$			$6.93_{0.26}$	
SPEA-II		$2.82_{1.27}$			$12.26_{4.10}$			$6.84_{0.41}$	
MORS		$2.54_{0.91}$			$7.39_{1.19}$			$6.19_{0.04}$	

Table 5.12: Median and Interquartile Range for the EP. Metric using Thumb-Parameters

	n = 20	n = 200	n = 2,000
ESPEA	$0.65_{0.09}$	$1.00_{0.31}$	$1.05_{0.05}$
MOMBI-II	$0.78_{0.06}$	$0.88_{0.06}$	$1.00_{0.01}$
NSGA-II	$0.56_{0.06}$	$0.81_{0.18}$	$0.91_{0.07}$
SPEA-II	$0.60_{0.08}$	$0.82_{0.09}$	$0.84_{0.06}$
MORS	$0.68_{0.11}$	$0.83_{0.04}$	$0.88_{0.02}$

Table 5.13: Median and Interquartile Range for the SPREAD Metric using Thumb-Parameters

	n = 20	n = 200	n = 2,000
ESPEA	$8.00e+02_{6.3e+02}$	$3.83e+05_{1.3e+06}$	$1.68e+05_{1.5e+05}$
MOMBI-II	$1.59e+02_{1.5e+02}$	$3.29e+04_{5.2e+04}$	$1.72e+03_{3.9e+02}$
NSGA-II	$1.25e+03_{1.2e+03}$	$1.13e+06_{2.2e+06}$	$6.93e+05_{5.0e+05}$
SPEA-II	$9.85e+02_{1.3e+03}$	$4.45e+05_{1.1e+06}$	$1.04e+06_{7.0e+05}$
MORS	$1.61e+02_{4.2e+02}$	$1.37e+05_{7.9e+04}$	$2.57e+04_{1.2e+04}$

Table 5.14: Median and Interquartile Range for the HV. Metric using Thumb-Parameters

the SOA. There is not a clear best SOA as PT and GSA can considered the best according to each of the rankings, although they remain very close to SORS and GGA.

Both rankings show an overall better performance for the MOA, although, these metrics are not able to determine a best algorithm among themselves, since their position in the ranking is very close.

Algorithm	Ranking
GGA_{WS}	27.66
GSA_{WS}	28.66
ES_{WS}	25.33
NES_{WS}	27.00
LS_{WS}	19.00
RD_{WS}	16.00
PT_{WS}	23.66
$SORS_{WS}$	21.00
GGA_{CHE}	9.00
GSA_{CHE}	8.66
ES_{CHE}	12.33
NES_{CHE}	11.66
LS_{CHE}	14.00
RD_{CHE}	15.66
PT_{CHE}	8.00
$SORS_{CHE}$	8.66
GGA_{EPS}	22.00
GSA_{EPS}	23.33
ES_{EPS}	20.66
NES_{EPS}	17.66
LS_{EPS}	14.33
RD_{EPS}	12.66
PT_{EPS}	16.00
$SORS_{EPS}$	17.00
ESPEA	3.66
MOMBI-II	3.66
NSGA-II	3.66
SPEA-II	2.66
MORS	1.33

Table 5.15: Friedman Ranking on IGD+ metric using Thumb-Parameters
Considering reduction performance (distributed according to chi-square with 28 degrees of freedom: 74.63).

Algorithm	Ranking
GGA_{WS}	27.00
GSA_{WS}	29.00
ES_{WS}	25.66
NES_{WS}	26.66
LS_{WS}	19.00
RD_{WS}	16.33
PT_{WS}	23.99
$SORS_{WS}$	20.66
GGA_{CHE}	8.33
GSA_{CHE}	7.66
ES_{CHE}	12.66
NES_{CHE}	11.66
LS_{CHE}	13.66
RD_{CHE}	15.33
PT_{CHE}	8.00
$SORS_{CHE}$	8.66
GGA_{EPS}	23.00
GSA_{EPS}	23.33
ES_{EPS}	20.66
NES_{EPS}	17.66
LS_{EPS}	14.00
RD_{EPS}	12.33
PT_{EPS}	17.66
$SORS_{EPS}$	17.00
ESPEA	3.66
MOMBI-II	3.66
NSGA-II	3.66
SPEA-II	2.66
MORS	1.33

Table 5.16: Friedman Ranking on EP. metric using Thumb-Parameters
Considering reduction performance (distributed according to chi-square with 28 degrees of freedom: 76.24).

The rankings for SPREAD and HV metrics are shown in Tables 5.17 and 5.14 respectively. For the Friedman test on the HV metric the higher is the better result. The best algorithms according to this ranking are SPEA-II and NSGA-II separated only by a small difference. SPEA-II shows a lower SPREAD metric which means their solutions are more convergent.

Algorithm	Ranking
ESPEA	4.33
MOMBI-II	3.66
NSGA-II	2.66
SPEA-II	1.99
MORS	2.33

Table 5.17: Friedman Ranking on SPREAD metric using Thumb Parameters
Considering reduction performance (distributed according to chi-square with 4 degrees of freedom: 4.53).

Algorithm	Ranking
ESPEA	3.33
MOMBI-II	1.00
NSGA-II	4.66
SPEA-II	4.00
MORS	2.00

Table 5.18: Friedman Ranking on HV metric using Thumb Parameters

In this section we tested both SOA and MOA using Thumb-Parameters, which involve the mutation rate and the crossover among the different algorithms.

5.4.3 Second Phase

5.4.4 Stability evaluation

To test the stability, an experiment took place measuring the results from the previous set of exploratory experiments for the $n = 20$ problem. The results were evaluated using the methodology described in Section 3.3. The number of stable students is considered for each preference criteria. And by each of the tested algorithms. The analysis was made using a metaheuristic LS, the single objective genetic algorithm GGA and the multi-objective algorithm NSGA-II. Using a random solution for comparison.

	$Gs - Stable$	$Ps - Stable$	$Int - Stable$	$Lvl - Stable$
Random Solution	3	0	0	12
Local Search	6	3	0	7
Genetic Generational Algorithm	8	0	2	7
NSGA-II	3	2	2	9

Table 5.19: Number of stable students in the groups by preference criteria

This analysis was made not to test which algorithm lead to the most stability, but to ensure that by optimising the objective functions it can lead to higher stability. However, in future research, it might be possible to have a deeper analysis of the behaviour of the stability of this problem.

5.5 Correlation between the results

After extensive analysis and looking at the results and observing an apparent trade-off between some of the objective functions There was the need of analysing the correlation among objectives over the non-dominated, which were taken from the best front obtained from the $n = 200$ problem.

Objective Functions	Statistic	P-value
Group Size and Participation Style	0.50	$2.17e - 08$
Group Size and Level	-0.96	$2.04e - 17$
Group Size and Interests Similarity	-0.57	$4.57e - 11$
Participation Style and Level	-0.55	$3.26e - 10$
Participation Style and Interests Similarity	0.03	0.74
Level and Interests Similarity	0.69	$2.04e - 17$

Table 5.20: Results of the Pearson correlation analysis

Taking the results from Table 5.20, The null Hypothesis of being two independent sets is rejected in all the comparisons except for the Ps and the Int functions using a less than one per cent significance. This means All the objective functions are correlated except for these two. However, for the Gs in respect to the Lvl and Int, there seems to be a negative correlation, the same with the Ps and Lvl. This was also observed visually in Figure **??** and others, where SOA seem to achieve better results for the Lvl and MOA better results for the Ps function.

It also makes sense for the Gs to be correlated to the Ps since the more members are in a group since the more people are in the group, the more participation will take place in the group. Gs and Lvl are negatively correlated, which also makes sense, since the higher the members of the group. This also validates more the use of MOA over SOA, since there are at least two objective functions which are not correlated.

Chapter 6

Conclusions and future Work.

6.1 Conclusion

The main aim of this work was to compare SOAs against MOAs for the stable student groups problem. It was shown that SOAs could outperform MOAs for a given a small enough problem; however, when the size of the problem increases, MOAs outperform SOAs and SOAs surpassing MOAs is not consistent. Although in the literature is common to find a reduction in the performance of MOAs for problems with more than three objective functions, known as many-objective optimisation problems, the results show that this was not the case for the problem, as the performance against SOAs remained consistent across the experiments.

The reason MOA had better performance may be attributed to its characteristics to escape local minima. This was shown by doing the experiments in Section 5.2.1, which had each objective function optimized individually, still MOA were able to outperform SOA.

The optimisation of objectives effectively leads to a stabilisation of the groups; however, it is still unknown if it can reach a perfectly stable configuration. Although this specific research only required near-stable solutions, so the results obtained are considered to satisfy this purpose.

RS stood out as the most competitive method; however, it got away from the reference front with the highest-sized problem. Although only tested in small-sized problems of $n = 20$ and $n = 200$, the last experimentation phase showed that NSGA-II could outperform RS using parameters related to the problem. This behaviour could be because the homogeneous distribution of the data-set, as most of its values have little variance.

It was shown that the way the problem was laid out had a high sensitivity to the starting point for the optimisations, which is mostly attributed to the way the neighbour solutions are considered. Each solution of a neighbourhood varies in a student belonging to a different group, which by the distribution of the synthetic dataset does not affect significantly the performance. There was a significant lack of novel solutions that improve the performance for the first two phases. RS, as well as PT, took advantage of the spread of starting points to come with novel solutions.

The reason why NSGA-II could perform better than the other algorithms can be attributed to a more aggressive mutation that overcame the lack of novel solutions. That makes sense since the SMP, which is a related problem, also considered an aggressive mutation to ensure that the algorithms covered the search space.

One additional advantage that MOA may have against SOA is the resulting solutions front have different solutions that optimise better a specific objective and in essence a different preference for the user. According to the exploration of stability, the student can give more importance to specific preferences. The ability to chose between groups within solutions that can include the student in a group that optimises their specific preference help improve the stability of the groups.

6.2 Future work

This work presents an effort to define stability for the stable student groups problem; however, this definition is not complete. There is a general idea of how stable the groups are and how to improve stability, but a generalisation that addresses any size of groups and their size restrictions, and a proof of whether it is always possible to find perfectly stable groups remains as an interesting opportunity to research in the future.

Although there was the possibility of optimising the stability metric in general, this was ignored in this research, because checking stability in higher problems to each group in a solution would cause higher complexity. However, if a general definition of stability can be determined that includes perfectly stable groups; thus, there might be additional heuristics to reach more stable groups. This also leads to a deeper exploring of the relationship between optimisation and stability.

Since the problem is sensitive to the starting point, another interesting path to explore consists in involving a clusterisation technique at the beginning to define the starting points. Then this can be compared to other clusterisation methods of multi-objective optimisation such as the work of Julia Handl [24].

Despite the results, there is still a chance for SOAs to outperform MOAs. Additional research may include further experiments with Chebyshev scalarization function using the ideal weights found, the same thing with the individual optimisation for the Pareto front construction.

It is suggested to test the same problem with relaxed constraints. For example, the group size function can be considered as a relaxed constraint that becomes more important as the solutions converge. Another area of improvement would be the parameter tuning for each of the algorithms, as this was only a brief exploration using a handful of algorithms. It can also be considered to use other configurations for the mutation and crossover operators, or even use other operators at all. Another thing to consider for the future is to test the algorithms

with a real dataset, as the distribution of the data was assumed, and, as it was shown, it could have lead to a negative impact in the results.

Bibliography

[1] Advertising on facebook.

[2] Demographic data 2018.

[3] AUDET, C., BIGEON, J., CARTIER, D., LE DIGABEL, S., AND SALOMON, L. Performance indicators in multiobjective optimization. *Optimization Online* (2018).

[4] BÄCK, T., FOGEL, D. B., AND MICHALEWICZ, Z. *Handbook of evolutionary computation*. CRC Press, 1997.

[5] BACK, T., HOFFMEISTER, F., AND SCHWEFEL, H.-P. A survey of evolution strategies. In *Proceedings of the fourth international conference on genetic algorithms* (1991), vol. 2, Morgan Kaufmann Publishers San Mateo, CA.

[6] BATTITI, R., AND PASSERINI, A. Brain–Computer evolutionary multiobjective optimization: A genetic algorithm adapting to the decision maker. *IEEE Trans. Evol. Comput. 14*, 5 (Oct. 2010), 671–687.

[7] BELLO, I., AND LIANSHUAN, S. Genetic algorithm for the stable marriage problem (smp). *International Journal of Science and Research 5*, 6 (2016), 939–944.

[8] BENNE, K. D., AND SHEATS, P. Functional roles of group members. *Group Facilitation*, 8 (2007), 30.

[9] BENNE, K. D., AND SHEATS, P. Functional roles of group members. *Group Facilitation*, 8 (2007), 30–35. Copyright - Copyright International Association of Facilitators 2007; Document feature - ; Last updated - 2010-06-07; SubjectsTermNotLitGenreText - United States–US.

[10] BOUSSAÏD, I., LEPAGNOT, J., AND SIARRY, P. A survey on optimization metaheuristics. *Information sciences 237* (2013), 82–117.

[11] BROOKS, S. H. A discussion of random methods for seeking maxima. *Operations Research 6*, 2 (1958), 244–251.

[12] COELLO, C. A. C., LAMONT, G. B., VAN VELDHUIZEN, D. A., ET AL. *Evolutionary algorithms for solving multi-objective problems*, vol. 5. Springer, 2007.

[13] COELLO, C. A. C., AND SIERRA, M. R. A study of the parallelization of a coevolutionary multi-objective evolutionary algorithm. In *Mexican International Conference on Artificial Intelligence* (2004), Springer, pp. 688–697.

[14] COOPER WILLIAM W., S. L. M., AND JOE, Z. *Non-Linear Optimization*, vol. 139. 2010.

[15] DE BEUKELAER, H., DAVENPORT, G. F., DE MEYER, G., AND FACK, V. James: An object-oriented java framework for discrete optimization using local search metaheuristics. *Software: Practice and Experience 47*, 6 (2017), 921–938.

[16] DEB, K., PRATAP, A., AGARWAL, S., AND MEYARIVAN, T. A fast and elitist multi-objective genetic algorithm: Nsga-ii. *IEEE transactions on evolutionary computation 6*, 2 (2002), 182–197.

[17] DONG, W., LEPRI, B., PIANESI, F., AND PENTLAND, A. Modeling functional roles dynamics in small group interactions. *IEEE transactions on multimedia 15*, 1 (2012), 83–95.

[18] DURILLO, J. J., AND NEBRO, A. J. jmetal: A java framework for multi-objective optimization. *Advances in Engineering Software 42*, 10 (2011), 760–771.

[19] EARL, D. J., AND DEEM, M. W. Parallel tempering: Theory, applications, and new perspectives. *Phys. Chem. Chem. Phys. 7* (2005), 3910–3916.

[20] EIBEN, A. E., SMITH, J. E., ET AL. *Introduction to evolutionary computing*, vol. 53. Springer, 2003.

[21] EMMERICH, M. T., AND DEUTZ, A. H. A tutorial on multiobjective optimization: fundamentals and evolutionary methods. *Natural Computing 17*, 3 (2018), 585–609.

[22] FONSECA, C. M., FLEMING, P. J., ET AL. Genetic algorithms for multiobjective optimization: Formulationdiscussion and generalization. In *Icga* (1993), vol. 93, Citeseer, pp. 416–423.

[23] GOLDBERG, D. E., AND HOLLAND, J. H. Genetic algorithms and machine learning. *Machine learning 3*, 2 (1988), 95–99.

[24] HANDL, J., AND KNOWLES, J. *Multi-Objective Clustering and Cluster Validation*. Springer Berlin Heidelberg, Berlin, Heidelberg, 2006, pp. 21–47.

[25] HERNÁNDEZ GÓMEZ, R., AND COELLO COELLO, C. A. Improved metaheuristic based on the r2 indicator for many-objective optimization. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation* (2015), ACM, pp. 679–686.

[26] HIRCHE, J. Dixon, l. c. w. / szegö, g. p. (eds.), towards global optimisation. proceedings of a workshop at the university of cagliari, italy, october 1974. amsterdam-oxford. north-holland publ. co. 1975. x. 472 s. *ZAMM - Journal of Applied Mathematics and*

Mechanics / Zeitschrift für Angewandte Mathematik und Mechanik 59, 2 (1979), 137–138.

[27] HUNING, A. Evolutionsstrategie. optimierung technischer systeme nach prinzipien der biologischen evolution, 1976.

[28] ISHIBUCHI, H., MASUDA, H., AND NOJIMA, Y. A study on performance evaluation ability of a modified inverted generational distance indicator. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation* (2015), ACM, pp. 695–702.

[29] ISHIBUCHI, H., MASUDA, H., TANIGAKI, Y., AND NOJIMA, Y. Difficulties in specifying reference points to calculate the inverted generational distance for many-objective optimization problems. In *2014 IEEE Symposium on Computational Intelligence in Multi-Criteria Decision-Making (MCDM)* (Dec 2014), pp. 170–177.

[30] ISHIBUCHI, H., NOJIMA, Y., AND TSUTOMU DOI. Comparison between single-objective and multi-objective genetic algorithms: Performance comparison and performance measures. In *2006 IEEE International Conference on Evolutionary Computation* (July 2006), pp. 1143–1150.

[31] IWAMA, K., AND MIYAZAKI, S. A survey of the stable marriage problem and its variants. In *International Conference on Informatics Education and Research for Knowledge-Circulating Society (icks 2008)* (Jan 2008), pp. 131–136.

[32] JIAO, R., ZENG, S., ALKASASSBEH, J. S., AND LI, C. Dynamic multi-objective evolutionary algorithms for single-objective optimization. *Appl. Soft Comput. 61* (2017), 793–805.

[33] JIN, N., AND RAHMAT-SAMII, Y. Advances in particle swarm optimization for antenna designs: Real-Number, binary, Single-Objective and multiobjective implementations. *IEEE Trans. Antennas Propag. 55*, 3 (Mar. 2007), 556–567.

[34] LEAL, P. B., SAVI, M. A., AND HARTL, D. J. Aero-structural optimization of shape memory alloy-based wing morphing via a class/shape transformation approach. *Proceedings of the Institution of Mechanical Engineers, Part G: Journal of Aerospace Engineering 232*, 15 (2018), 2745–2759.

[35] LIN, W. Y., LEE, W. Y., AND HONG, T. P. Adapting crossover and mutation rates in genetic algorithms. *Journal of Information Science and Engineering 19*, 5 (2003), 889–903.

[36] MADYLOVA, A., AND OGUDUCU, S. G. A taxonomy based semantic similarity of documents using the cosine measure. In *2009 24th International Symposium on Computer and Information Sciences* (Sept 2009), pp. 129–134.

[37] MATSUYAMA, Y., AKIBA, I., FUJIE, S., AND KOBAYASHI, T. Four-participant group conversation: A facilitation robot controlling engagement density as the fourth participant. *Computer Speech & Language 33*, 1 (9 2015), 1–24.

[38] McCowan, I., Carletta, J., Kraaij, W., Ashby, S., Bourban, S., Flynn, M., Guillemot, M., Hain, T., Kadlec, J., Karaiskos, V., et al. The ami meeting corpus. In *Proceedings of the 5th International Conference on Methods and Techniques in Behavioral Research* (2005), vol. 88, p. 100.

[39] Mehboob, U., Qadir, J., Ali, S., and Vasilakos, A. Genetic algorithms in wireless networking: techniques, applications, and issues. *Soft Computing 20*, 6 (2016), 2467–2501.

[40] Mitchell, M., Holland, J. H., and Forrest, S. When will a genetic algorithm outperform hill climbing. In *Advances in neural information processing systems* (1994), pp. 51–58.

[41] Nomura, T. An analysis on crossovers for real number chromosomes in an infinite population size. In *IJCAI (2)* (1997), pp. 936–941.

[42] Reynolds, R. G., Kohler, T. A., and Kobti, Z. The effects of generalized reciprocal exchange on the resilience of social networks: An example from the prehispanic mesa verde region. *Computational & mathematical organization theory 9*, 3 (2003), 227–254.

[43] Riquelme, N., Von Lücken, C., and Baran, B. Performance metrics in multi-objective optimization. In *2015 Latin American Computing Conference (CLEI)* (2015), IEEE, pp. 1–11.

[44] Seyed Shirkhorshidi, A., Aghabozorgi, S., and Wah, Y. A Comparison Study on Similarity and Dissimilarity Measures in Clustering Continuous Data.

[45] Shiau, T. N., Kuo, C. P., and Hwang, J. R. Multiobjective optimization of a flexible rotor in magnetic bearings with critical speeds and control current constraints. In *ASME 1994 International Gas Turbine and Aeroengine Congress and Exposition* (1994), American Society of Mechanical Engineers, pp. V005T14A039–V005T14A039.

[46] Sung-Hyuk Cha. Comprehensive Survey on Distance/Similarity Measures between Probability Density Functions. Tech. rep.

[47] Teich, J. Pareto-front exploration with uncertain objectives. In *International Conference on Evolutionary Multi-Criterion Optimization* (2001), Springer, pp. 314–328.

[48] Trautmann, H., Wagner, T., and Brockhoff, D. R2-emoa: Focused multiobjective search using r2-indicator-based selection. In *International Conference on Learning and Intelligent Optimization* (2013), Springer, pp. 70–74.

[49] Van Veldhuizen, D. A., and Lamont, G. B. Evolutionary computation and convergence to a pareto front. In *Late breaking papers at the genetic programming 1998 conference* (1998), pp. 221–228.

[50] Vinciarelli, A., Valente, F., Harsha Yella, S., and Sapru, A. Understanding Social Signals in Multi-party Conversations: Automatic Recognition of Socio-Emotional Roles in the AMI Meeting Corpus.

[51] WANG, N. F., AND TAI, K. Target matching problems and an adaptive constraint strategy for multiobjective design optimization using genetic algorithms. *Comput. Struct. 88*, 19 (Oct. 2010), 1064–1076.

[52] ZABINSKY, Z. B. *Stochastic adaptive search for global optimization*, vol. 72. Springer Science & Business Media, 2013.

[53] ZHANG, Q., AND LI, H. Moea/d: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on evolutionary computation 11*, 6 (2007), 712–731.

[54] ZITZLER, E., LAUMANNS, M., AND THIELE, L. Spea2: Improving the strength pareto evolutionary algorithm. *TIK-report 103* (2001).

[55] ZITZLER, E., AND THIELE, L. Multiobjective optimization using evolutionary algorithms—a comparative case study. In *International conference on parallel problem solving from nature* (1998), Springer, pp. 292–301.

[56] ZITZLER, E., AND THIELE, L. Multiobjective evolutionary algorithms: a comparative case study and the strength pareto approach. *IEEE transactions on Evolutionary Computation 3*, 4 (1999), 257–271.

[57] ZITZLER, E., THIELE, L., LAUMANNS, M., FONSECA, C. M., AND DA FONSECA, V. G. Performance assessment of multiobjective optimizers: an analysis and review. *IEEE Transactions on Evolutionary Computation 7*, 2 (April 2003), 117–132.

Curriculum Vitae

(Only one page) Master student was born in Monterrey, México, on December 25, 2208. He earned the Systems Engineering degree from the Instituto Tecnológico y de Estudios Superiores de Monterrey, Monterrey Campus in May 2008. He was accepted in the graduate programs in Information Technologies and Electronics in May 2010.

This document was typed in using LaTeX 2_ε[a] by Miguel Ángel Bravo Vidales.

[a]The style file `phdThesisFormat.sty` used to set up this thesis was prepared by the Center of Intelligent Systems of the Instituto Tecnológico y de Estudios Superiores de Monterrey, Monterrey Campus